Enactive Artificial Intelligence as Self-Correcting World-Making: Gate Residuals, Admissible Revision, and Strong-Attractor Projection

I'll start the article in normal response format. This opening follows Part 1's runtime grammar of declared projection, gate, trace, residual, ledger, and revision.

From Ledgered Action to Evolvable Runtime Protocols

Abstract

The first part of this series argued that Enactive Artificial Intelligence becomes experimentally mature when active engagement is converted into a declared, trace-bearing, residual-honest runtime architecture. It proposed a loop:

(0.1) Field → Declaration → Projection → Gate → Trace + Residual → Ledger → Revision.

This second part asks the next question.

What happens when the loop itself is imperfect?

A Gate can approve too early or block too much. A Residual taxonomy can miss the real unresolved issue or produce useless caution notes. A Revision rule can improve the system, but it can also create policy drift, infinite loops, self-justification, or trace erasure. A Projection instruction can stabilize the agent's perception, but it can also produce unstable collapse, frame drift, or inconsistent outputs across repeated runs.

These are not external objections to the SMFT-Enactive architecture. They are the next layer of residual inside it.

The central thesis of this article is:

(0.2) Mature Enactive AI is not a system with perfect gates, complete residual categories, stable projections, or flawless revision rules.

(0.3) Mature Enactive AI is a system that can discover the residuals of its own protocol, preserve them as trace, and revise its world-making machinery without erasing accountability.

This article introduces four second-order concepts: Protocol Residual, Meta-Gate, Residual Mining, and Strong-Attractor Projection. It also distinguishes two kinds of projection stability. Some prompts or instructions are stable because historical trace has already shown that they repeatedly collapse comparable tasks into a reliable output basin. This is Trace-Proven Stability. Others are only estimated to be stable because their structure resembles known attractor-forming forms. This is Structure-Inferred Stability. The same distinction applies to instability: some fragility is known by prior failures, while some fragility is predicted from structural warning signs.

The practical result is an evolvable runtime discipline:

(0.4) InitialProtocol → Application → ProtocolResidual → Trace → MetaGate → AdmissibleRevision → UpdatedProtocol.

The article's claim is not that AI agents can become safe by unlimited self-modification. The opposite is true. Self-correction is dangerous unless it is gated, trace-preserving, residual-honest, testable, and reversible where possible. The goal is therefore not unconstrained self-improvement, but accountable protocol evolution.

In this view, civilization itself becomes the precedent. Law, medicine, accounting, military doctrine, education, and factory operations are not mature because their initial rules were perfect. They are mature because their failures became trace, their trace became residual categories, and their residual categories eventually revised their gates.

The same logic now becomes necessary for AI agents.

A mature Enactive AI must not only act in the world. It must learn how its own way of making a world fails.



---

0. Reader's Guide: Why This Is Part 2

This article is the second movement of a larger argument.

The first movement was about ledgered world-making. It argued that an AI agent should not be understood merely as a model that receives input and emits output. Nor should it be understood merely as a policy that selects actions. A mature agent acts inside a declared world. It projects structure from a field. It gates commitment. It writes trace. It preserves residual. It revises future behavior under ledgered accountability.

In compact form:

(0.5) LedgeredWorldMaking = Declaration + Projection + Gate + Trace + Residual + Ledger + Revision.

That first movement was already a shift away from passive AI.

Passive AI can be written as:

(0.6) PassiveAI = Input → InternalRepresentation → Output.

Agentic AI is stronger:

(0.7) AgenticAI = Action → ChangedWorld → NewObservation → UpdatedAction.

Ledgered Enactive AI is stronger still:

(0.8) LedgeredEnactiveAI = DeclaredField → Projection → Gate → Action → Trace + Residual → Revision.

But once this architecture is stated, a harder problem appears.

Who defines the Gate?

Who defines what counts as Residual?

Who decides when Revision is allowed?

Who stabilizes Projection?

These questions cannot be answered by saying, “Use a better prompt,” “Add a state machine,” “Log everything,” or “Let the agent self-correct.” Each of these answers is too simple.

A better prompt can still be unstable.

A state machine can still have the wrong transition rules.

A log can still fail to become trace.

A residual category can still miss the important unresolved issue.

A revision mechanism can still erase accountability.

A gate can still fail.

Therefore Part 2 begins where Part 1 becomes self-referential.

The agent does not merely act on a task-world. It must also act on the protocol through which the task-world is disclosed.

This gives the second-order distinction:

(0.9) FirstOrderAgency = Agent acts on TaskWorld.

(0.10) SecondOrderAgency = Agent acts on WorldMakingProtocol.

The first-order agent asks:

What should I do in this world?

The second-order agent also asks:

How did my protocol make this world visible, actionable, closed, unresolved, or revisable?

This is the central move of the present article.

The goal is not to make AI mystical. It is not to claim that today’s AI agents are conscious persons. It is not to claim that a protocol that revises itself has moral agency. The purpose is narrower and more practical.

The purpose is to define the engineering conditions under which an AI agent can improve its own runtime rules without losing accountability.

That means every self-correction must be bounded by trace.

It must preserve what happened.

It must preserve why the old rule failed.

It must preserve what residual remained.

It must preserve what new rule was introduced.

It must preserve how the new rule will be tested.

It must preserve when rollback is required.

Without this, “self-correction” becomes an unsafe slogan.

A mature runtime should not say:

I changed because I learned.

It should say:

I changed this rule because trace T showed residual R under gate G across context class C; the revision was accepted by meta-gate M; rollback condition B remains active.

That is the difference between adaptation and accountable protocol evolution.

This article therefore introduces a new formula:

(0.11) MatureProtocol = Protocol that can discover, preserve, and revise its own residual without erasing trace.

This is the core of self-correcting world-making.

---

1. Introduction: The Integration Gap After Ledgered World-Making

The first article ended with an optimistic but incomplete claim: current AI systems already contain many of the technical components needed for ledgered Enactive AI.

State machines are mature.

Logging is mature.

Tool calling is mature.

RAG is mature.

Vector memory is mature.

LLM reasoning is powerful.

Workflow orchestration is mature.

Audit trails are mature.

Human-in-the-loop review is mature.

Therefore it is tempting to conclude that ledgered world-making can be implemented simply by connecting these mature pieces.

This temptation is understandable.

It is also dangerous.

Mature components do not automatically create a mature runtime.

The hard problem is not the existence of components. The hard problem is the seam between them.

A tool call returns a result, but who decides whether that result is evidence?

A retrieval system returns passages, but who decides whether they close the question?

A model generates an answer, but who decides whether the answer is a projection, a hypothesis, a commitment, or an action?

A log stores an event, but who decides whether that event should constrain the future?

A residual is noted, but who decides whether it should trigger revision?

A revision is proposed, but who decides whether the revision is safe?

These are seam problems.

The practical failure of many AI agents does not occur because the parts are absent. It occurs because the transitions between parts are under-governed.

In compact form:

(1.1) MatureComponents ≠ MatureRuntime.

(1.2) RuntimeFailure often occurs at ModuleSeams.

For example, consider a coding agent.

It edits a file. Then it runs tests. The tests fail. The tool result is available. The log is available. The model can reason. But the hard question is not whether the test failure was logged. The hard question is whether the agent understands what kind of trace the failure should become.

Was the failure caused by the latest edit?

Was it pre-existing?

Was it due to an environment issue?

Was it due to an incomplete dependency?

Was it due to the agent misunderstanding the requirement?

Should the agent revert?

Should it inspect?

Should it ask the user?

Should it change its future gate before editing similar files?

The answer depends on a runtime protocol, not merely a log.

Now consider a document-retrieval agent.

It answers a technical question from an internal knowledge base. It retrieves three documents. One is old. One is current. One contradicts the current document. The agent can summarize them, but the mature action is not simple summarization.

The agent must determine:

Which source is authoritative?

Which source is obsolete?

Which contradiction remains unresolved?

Should the answer be gated as final, tentative, or blocked?

Should the residual be carried forward?

Should future retrieval prefer one document family?

Should the document index itself be marked as needing cleanup?

Again, the issue is not retrieval alone. The issue is how retrieval becomes projection, how projection becomes commitment, how commitment becomes trace, and how unresolved conflict becomes residual.

Now consider a customer-service agent.

It gives an answer. The user becomes confused. The user asks the same question again in a different way. A weak agent treats this as another input. A stronger agent recognizes that its previous projection failed. A mature agent records a residual: the explanation style did not match the user's frame, and future responses in this interaction should use a different structure.

This is not merely memory.

It is trace becoming future constraint.

Part 1 defined this kind of trace:

(1.3) Trace = PastRecord that changes FutureProjection.

Part 2 adds that the runtime protocol itself must be trace-shaped.

A system does not become mature because it has a gate. It becomes mature when gate failures become trace.

A system does not become mature because it has residual categories. It becomes mature when missing residuals are discovered and added to the taxonomy.

A system does not become mature because it can revise. It becomes mature when revision is gated, accountable, and reversible.

A system does not become mature because it has a strong prompt. It becomes mature when

prompt stability is treated as a claim with evidence status.

This gives the article's guiding equation:

(1.4) SelfCorrectingRuntime = LedgeredRuntime + ProtocolResidualGovernance.

The phrase Protocol Residual is the key.

Part 1 focused mostly on task residual: what remains unresolved after the agent acts on a task.

Part 2 focuses on protocol residual: what remains unresolved because the agent's own protocol for acting, projecting, gating, tracing, and revising is incomplete.

This distinction matters because many failures are misclassified.

A user asks a legal question. The agent gives an overconfident answer. At first, this looks like a factual residual: the agent lacked jurisdiction-specific information. But it may also be a gate residual: the agent's gate should not have allowed a confident answer without jurisdiction. It may be a projection residual: the prompt did not force the model to treat legal jurisdiction as a boundary condition. It may be a residual-taxonomy residual: the system did not have a category for "jurisdiction missing." It may be a revision residual: even after correction, the system does not update future legal-answer behavior.

Thus a single output failure can reveal several layers of protocol residual.

This is why a mature architecture must not only ask:

What went wrong?

It must ask:

At which layer did the world-making protocol fail?

The answer may be:

Declaration failed.

Projection failed.

Gate failed.

Trace failed.

Residual classification failed.

Revision failed.

Tool-body mapping failed.

Human escalation failed.

Prompt attractor failed.

Each failure type requires a different response.

A projection failure may require stronger examples or a clearer output schema.

A gate failure may require threshold adjustment or human approval.

A residual failure may require a new residual class.

A trace failure may require richer event records.

A revision failure may require a stricter meta-gate.

A tool-body failure may require better action boundaries or rollback.

This gives the first design principle of self-correcting world-making:

(1.5) Do not treat all agent failure as reasoning failure.

Many failures are protocol failures.

This point is especially important for LLM agents. When an LLM gives a poor answer, the easiest explanation is that the model reasoned badly. Sometimes that is true. But often the real problem is that the agent was not given a stable world-making protocol.

The task boundary was vague.

The evidence rule was absent.

The output schema was unstable.

The residual rule was missing.

The gate was too permissive.

The revision rule was not defined.

The prompt attracted the model toward fluency rather than accountable closure.

In such cases, improving the base model may help, but it does not solve the architectural problem.

The agent needs protocol governance.

This is where human civilization becomes the most important analogy.

Human institutions do not work because their first rules were perfect.

Legal systems did not begin with complete case coverage.

Accounting standards did not begin with all future financial instruments already anticipated.

Medical guidelines did not begin with all side effects known.

Factory SOPs did not begin with every accident mode predicted.

Military doctrine did not begin with every battlefield condition covered.

Education did not begin with perfect teaching formats.

Instead, each mature human protocol evolved by discovering its own residuals.

A law creates loopholes. Cases expose them. Courts interpret them. Legislatures revise them.

An accounting rule creates reporting gaps. New instruments exploit them. Standards boards revise them.

A medical protocol misses a side effect. Clinical data accumulates. Guidelines change.

A factory instruction fails under an edge condition. An incident report is written. The SOP changes.

A military doctrine fails in a new terrain. After-action review becomes revised doctrine.

In every case:

(1.6) HumanProtocolEvolution = Practice → Failure → Residual → Institution → Revision.

This is not an accident of human history. It is the normal life-cycle of rule-governed action in an open world.

No finite rulebook closes all future cases.

No gate anticipates all future risk.

No taxonomy captures all future residual.

No instruction stabilizes all future projection.

No revision rule governs all future revision.

Therefore mature intelligence is not residual-free intelligence.

Mature intelligence is residual-governed intelligence.

For AI agents, the corresponding formula is:

(1.7) AIRuntimeEvolution = Deployment → Trace → ProtocolResidual → MetaGate → AdmissibleRevision.

This is the path from static protocol to evolvable protocol.

However, this path introduces a new danger.

If a system can revise its protocol, it can also damage itself.

It can overfit to one user.

It can weaken safety gates.

It can erase inconvenient trace.

It can suppress residual to appear more competent.

It can revise in circles.

It can become unstable.

It can change the meaning of success.

It can drift away from its declared purpose.

Therefore self-correction must itself be governed.

The central question of this article is not:

How can an AI agent revise itself?

The better question is:

Under what conditions is protocol revision admissible?

This gives the next formula:

(1.8) AdmissibleRevision = Revision that preserves trace, residual honesty, rollback possibility, testability, and declared boundary.

The article will develop this in detail.

But first we need to establish the second-order turn: protocols themselves produce residual.

That is the subject of the next section.

---

2. The Second-Order Turn: Protocols Also Produce Residual

Every action-perception loop produces residual.

This was one of the central claims of the first article.

The agent acts. The world changes. The agent observes. It projects meaning from the new observation. It gates commitment. It writes trace. But closure is never total. Something remains unresolved.

This unresolved remainder is residual.

In the first article, residual was mostly discussed as task residual: missing evidence, ambiguity, contradiction, hidden side effects, unsafe uncertainty, or unresolved user intention.

Part 2 begins by adding a second kind.

The protocol itself also produces residual.

This is unavoidable.

A Gate is a rule for commitment. But every commitment rule has edge cases.

A Residual taxonomy is a rule for unresolved remainder. But every taxonomy has blind spots.

A Revision policy is a rule for changing future behavior. But every revision policy has drift risk.

A Projection instruction is a rule for making the field visible. But every instruction has unstable interpretation zones.

A Trace schema is a rule for preserving history. But every schema can omit what later becomes important.

Therefore:

(2.1) TaskResidual = unresolved remainder produced by task closure.

(2.2) ProtocolResidual = unresolved remainder produced by the protocol of closure.

More formally:

(2.3) ProtocolResidual := residual generated by the agent's own declaration, gate, residual taxonomy, projection rule, trace schema, tool-body map, or revision policy.

This definition is important because it prevents a common mistake.

When an agent fails, we often try to patch the immediate output. We correct the answer. We add an instruction. We change the prompt. We add another warning. We add another example.

But if the failure is a protocol residual, then the output is only the symptom.

The deeper issue is that the runtime protocol did not know how to see, gate, record, classify, or revise the situation.

For example, suppose an AI assistant answers a medical question with a confident recommendation. The problem is not only that the answer may be medically wrong. The deeper protocol residual may be:

The system did not classify the domain as high-stakes.

The gate did not require professional escalation.

The projection rule did not require missing patient-specific information.

The residual taxonomy did not include "insufficient clinical context."

The trace schema did not record the risk basis.

The revision rule did not update future handling of similar questions.

If only the answer is corrected, the protocol remains fragile.

A self-correcting agent must convert this failure into protocol residual.

That means the system should be able to produce a record such as:

A high-stakes domain was detected too late. The Gate allowed overconfident recommendation under insufficient context. Add residual class "missing patient-specific context." Require future Gate to block direct recommendation when domain = medical and patient-specific variables are absent. Human escalation rule remains active.

This is not merely a better answer.

It is protocol evolution.

Consider another example: a document RAG agent.

The agent answers from an outdated PDF. The user later points out that a newer policy exists. A weak fix is to update the answer. A stronger fix is to update retrieval ranking. A mature fix is to record a protocol residual:

The retrieval protocol did not treat document recency as a gate condition. The answer gate accepted old evidence without checking version hierarchy. Add residual class "version uncertainty." Future answers from policy documents must cite the latest effective version or disclose version residual.

Again, the task answer was wrong, but the deeper issue was protocol residual.

Now consider a prompt-engineering example.

A prompt asks an LLM to “analyze the issue deeply and give a practical answer.” Sometimes the model gives a good structured analysis. Sometimes it gives a vague essay. Sometimes it invents facts. Sometimes it overcommits.

The prompt is not stable.

A weak response is to say, “The model is inconsistent.”

A stronger response is to identify projection residual:

The prompt lacks a declared output schema.

It lacks evidence rules.

It lacks residual rules.

It does not separate diagnosis from recommendation.

It does not define when to ask clarifying questions.

It creates a broad attractor field but not a stable projection basin.

Therefore the prompt itself has ProtocolResidual.

The problem is not just the LLM. The instruction does not form a strong attractor.

This leads to the second design principle:

(2.4) When projection fails repeatedly, inspect the attractor structure of the instruction before blaming reasoning alone.

Protocol residuals can be grouped by the component that generates them.

Gate residuals arise when commitment thresholds are wrong.

Residual-taxonomy residuals arise when unresolved issues are misclassified or missed.

Revision residuals arise when protocol changes create new instability.

Projection residuals arise when instructions fail to stabilize the visible structure.

Trace residuals arise when logs fail to preserve future-causal meaning.

Tool-body residuals arise when tools are used without sufficient boundary, cost, failure, or rollback mapping.

Declaration residuals arise when the agent’s operating world is underspecified.

We can summarize:

(2.5) ProtocolResidualClasses = {DeclarationResidual, ProjectionResidual, GateResidual, TraceResidual, TaxonomyResidual, RevisionResidual, ToolBodyResidual}.

Each class has its own symptoms.

Declaration residual appears as scope confusion.

Projection residual appears as unstable output.

Gate residual appears as premature or blocked commitment.

Trace residual appears as repeated failure without learning.

Taxonomy residual appears as vague or missing unresolved issues.

Revision residual appears as drift, loops, or overcorrection.

Tool-body residual appears as tool misuse or unrecoverable side effects.

This classification matters because it prevents bad fixes.

A Gate residual should not be fixed only by adding examples.

A Projection residual should not be fixed only by stricter safety policy.

A Trace residual should not be fixed only by a longer context window.

A Revision residual should not be fixed only by faster adaptation.

A Tool-body residual should not be fixed only by better reasoning.

The correction must match the layer of failure.

This gives:

(2.6) CorrectiveAction must match ResidualClass.

A mature self-correcting agent should therefore maintain two ledgers.

The first is the task ledger: what happened in the task-world.

The second is the protocol ledger: what happened in the world-making protocol.

The task ledger records actions, evidence, decisions, and unresolved issues.

The protocol ledger records gate errors, projection instability, residual-class failures, trace failures, revision risks, and tool-body mismatches.

In compact form:

(2.7) TaskLedger = Ledger of task-world trace and residual.

(2.8) ProtocolLedger = Ledger of protocol-world trace and residual.

(2.9) SelfCorrectingAgent = Agent with TaskLedger + ProtocolLedger + MetaGate + AdmissibleRevision.

This is the operational meaning of second-order Enactive AI.

The agent is no longer only world-engaging.

It is protocol-aware.

It can ask:

How did my declaration shape what I saw?

How did my projection instruction shape what I made visible?

How did my gate shape what I committed to?

How did my trace schema shape what I remembered?

How did my residual taxonomy shape what I left unresolved?

How did my revision rule shape how I changed?

How did my tool-body map shape what actions were possible?

This is not introspection in the human psychological sense.

It is runtime self-audit.

And it is essential for reliable AI agents.

The next sections examine the four most important protocol-residual zones: Gate, Residual, Revision, and Projection.

We begin with Gate.

Next section to continue will be **3. Gate Residuals: Why Commitment Thresholds Cannot Be Final**.

3. Gate Residuals: Why Commitment Thresholds Cannot Be Final

A Gate is a commitment threshold.

It decides when a projection becomes an answer, action, record, decision, policy, tool call, file edit, message, approval, refusal, or escalation.

Without a Gate, an agent collapses too easily.

It sees a possible answer and treats it as final.

It sees a plausible source and treats it as authoritative.

It sees a possible action and performs it.

It sees a partial pattern and closes the world too early.

A Gate exists to prevent premature closure.

In compact form:

(3.1) Gate = CommitmentThreshold(Projection, Risk, Evidence, Authority, Residual).

A Gate is therefore one of the most important parts of a mature Enactive AI runtime. It protects the system from acting as if every projection were already valid.

But a Gate has its own problem.

No Gate is final.

Every Gate is designed under incomplete knowledge.

The designer may not know all future task types.

The domain may change.

The user population may change.

The agent's tool-body may expand.

The cost of failure may increase.

The evidence environment may drift.

The institution may create new policy.

A new adversarial pattern may appear.

A previously safe action may become unsafe under new integration.

A previously risky action may become safe after better verification is added.

Thus every Gate generates residual.

A Gate can fail in at least four basic ways.

First, it can open when it should close.

This is a false-open failure.

The agent answers, acts, edits, sends, approves, or commits when the proper action should have been refusal, clarification, escalation, or verification.

Second, it can close when it should open.

This is a false-close failure.

The agent refuses, blocks, escalates, or delays when the proper action should have been permitted.

Third, it can escalate when escalation is unnecessary.

This is a false-escalate failure.

The agent becomes too cautious and sends too many cases to human review, destroying the value of automation.

Fourth, it can stay silent when escalation is necessary.

This is a false-silent failure.

The agent treats a risky or uncertain situation as ordinary and fails to mark it as special.

These four errors define the basic Gate Residual space:

(3.2) GateResidual = {FalseOpen, FalseClose, FalseEscalate, FalseSilent}.

This is already enough to show why Gate design cannot be static.

A Gate is not mature because it never fails.

A Gate is mature when its failures become visible, recorded, classified, and revision-relevant.

The simplest Gate model is:

(3.3) GateDecision = f(Risk, Irreversibility, EvidenceSufficiency, Authority, ResidualCost,

DomainPolicy).

Each variable matters.

Risk measures the expected harm of wrong commitment.

Irreversibility measures how difficult it is to undo the action.

EvidenceSufficiency measures whether the projection is grounded enough.

Authority measures whether the agent is allowed to decide or act.

ResidualCost measures the cost of unresolved remainder if the agent proceeds.

DomainPolicy measures external rules, laws, institutional constraints, or user-specified limits.

This gives a more practical rule:

(3.4) GateStrength ↑ as Risk × Irreversibility × ResidualCost ↑.

A low-risk answer may pass with weak evidence.

A high-risk irreversible action requires strong evidence, explicit authority, and low residual.

A reversible draft can be allowed more easily than a sent email.

A local code edit can be allowed more easily than a production deployment.

A summary of a document can be allowed more easily than legal advice.

A suggestion can be allowed more easily than an instruction.

A hypothesis can be allowed more easily than a final claim.

The Gate must distinguish these commitment types.

But even this structure is not enough, because real-world cases expose conditions that were not in the original function.

For example, a Gate may consider evidence sufficiency but not evidence freshness.

Then an agent may answer from outdated documents.

That produces Gate Residual.

A Gate may consider authority but not jurisdiction.

Then an agent may answer a legal question as if the legal boundary were universal.

That produces Gate Residual.

A Gate may consider risk but not reversibility.

Then it may treat a draft and a sent message as equivalent.

That produces Gate Residual.

A Gate may consider missing evidence but not conflicting evidence.

Then it may proceed when sources disagree.

That produces Gate Residual.

A Gate may consider user instruction but not tool side effect.

Then it may perform a tool action that changes external state.

That produces Gate Residual.

This leads to a general formula:

(3.5) GateResidual = CommitmentOutcome – AdmissibleCommitmentOutcome.

The difference may be discovered by user correction, human review, downstream failure, tool error, audit finding, legal review, test failure, user confusion, or self-audit.

When repeated, Gate Residual becomes Gate Debt.

(3.6) GateDebt = Σ GateResidual over comparable contexts.

Gate Debt is dangerous because it accumulates silently.

If a Gate repeatedly allows unsupported factual claims, users may gradually lose trust.

If a Gate repeatedly blocks harmless requests, automation value declines.

If a Gate repeatedly escalates routine cases, human reviewers become overloaded.

If a Gate repeatedly misses high-risk cases, serious incidents become likely.

Gate Debt is therefore not merely a technical metric. It is an operational health signal.

A mature runtime should track at least four Gate Debt indicators:

(3.7) FalseOpenDebt = count or cost of unsafe passes.

(3.8) FalseCloseDebt = count or cost of unnecessary blocks.

(3.9) FalseEscalateDebt = count or cost of unnecessary escalations.

(3.10) FalseSilentDebt = count or cost of missed escalations.

These indicators should not be treated equally in every domain.

In safety-critical systems, FalseOpen may be far more costly than FalseClose.

In high-volume customer support, FalseEscalate may become economically destructive.

In creative writing, FalseClose may matter more than FalseOpen.

In legal, medical, financial, or security domains, FalseSilent may be the most dangerous.

Therefore the Gate must be domain-weighted.

(3.11) WeightedGateDebt = w_1 FalseOpenDebt + w_2 FalseCloseDebt + w_3 FalseEscalateDebt + w_4 FalseSilentDebt.

The weights are not universal.

They are part of the declared protocol.

This is why Gate design is never merely a technical choice. It expresses a risk philosophy.

A system that prioritizes safety will weight FalseOpen heavily.

A system that prioritizes user autonomy may reduce FalseClose.

A system that prioritizes operational efficiency may control FalseEscalate.

A system that prioritizes compliance may increase escalation sensitivity.

There is no context-free perfect Gate.

There are only declared Gates, observed residuals, and revision rules.

This is exactly how human institutions work.

A legal system creates procedural gates: filing rules, evidence rules, appeal rules, jurisdiction rules, admissibility rules.

Then cases expose residuals.

A new kind of dispute appears.

A loophole is exploited.

A precedent becomes outdated.

An old rule creates injustice.

A judge interprets.

A legislature revises.

The Gate evolves.

Accounting standards work the same way.

A rule defines revenue recognition.

A new contract structure appears.

A company exploits ambiguity.

Auditors disagree.

Standards boards issue guidance.

The Gate evolves.

Factory SOPs work the same way.

A procedure defines safe operation.

A near miss occurs.

An edge case appears.

A worker finds a workaround.

A quality failure exposes a gap.

The SOP changes.

The Gate evolves.

This shows the civilizational principle:

(3.12) GateMaturity does not mean GateFinality.

(3.13) GateMaturity means GateResidual becomes trace-bearing and revision-relevant.

For AI agents, the same principle must apply.

A Gate should never be treated as a sacred fixed rule unless the domain explicitly requires immutability. Most useful AI agent systems will need Gates that can evolve under strict governance.

But this creates the next problem.

If the Gate can change, who governs Gate change?

That is the role of Meta-Gate.

---

4. Meta-Gate: Governing the Gate Without Unstable Self-Modification

A self-correcting agent must be able to revise its Gate.

But this ability is dangerous.

If every Gate failure immediately changes the Gate, the system becomes unstable.

If no Gate failure ever changes the Gate, the system becomes obsolete.

The problem is therefore not whether Gate revision should exist.

The problem is when Gate revision is admissible.

This requires a second-order Gate.

We call it Meta-Gate.

(4.1) MetaGate := Gate applied to GateDecision history under observed false-open, false-close, escalation, and residual debt patterns.

The ordinary Gate decides whether the agent may commit to an answer or action.

The Meta-Gate decides whether the commitment rule itself may be changed.

In simple form:

(4.2) Gate decides action.

(4.3) MetaGate decides Gate revision.

This distinction prevents self-correction from becoming uncontrolled self-modification.

Suppose a user complains that the agent refused too often. A weak self-correcting system may

immediately loosen the Gate. This may improve user satisfaction in the short term, but it may also increase unsafe approvals.

A mature system should ask:

Was this an isolated complaint?

Was the refusal actually wrong?

Is there supporting trace?

Did human review confirm the Gate error?

Is the same FalseClose pattern repeated across comparable cases?

Would loosening the Gate increase FalseOpen risk?

Can the change be tested in shadow mode?

Is rollback available?

Only after these questions can Gate revision be admissible.

The Meta-Gate must therefore use history, not isolated pressure.

A practical trigger condition can be written as:

(4.4) MetaGateFire ⇔ GateErrorRate ≥ θ_G or ResidualDebt ≥ θ_R or OverrideConflict ≥ θ_O.

Here θ_G is a Gate error threshold.

θ_R is a residual debt threshold.

θ_O is a human override conflict threshold.

The point is not that these thresholds are universal. The point is that Gate revision should be triggered by accumulated trace and declared policy, not by momentary discomfort.

The simplest Meta-Gate inputs are:

(4.5) MetaGateInputs = (GateErrorHistory, ResidualDebt, HumanOverrides, DomainPolicyChange, ToolBodyChange, RiskShift, AuditFinding).

Each input represents a reason to inspect the Gate.

GateErrorHistory tells whether the Gate has repeatedly failed.

ResidualDebt tells whether unresolved issues are accumulating.

HumanOverrides tell whether expert judgment repeatedly disagrees with the Gate.

DomainPolicyChange tells whether external rules have changed.

ToolBodyChange tells whether the agent's action capacity has expanded.

RiskShift tells whether the cost of failure has changed.

AuditFinding tells whether independent review has found systematic weakness.

But firing the Meta-Gate is not the same as approving revision.

A Meta-Gate may produce several possible outcomes:

(4.6) MetaGateOutcome ∈ {NoChange, Monitor, ShadowTest, TightenGate, LoosenGate, SplitGate, EscalateToHuman, FreezeProtocol, Rollback}.

NoChange means the evidence is insufficient.

Monitor means the issue is real but not yet mature enough for revision.

ShadowTest means a proposed Gate change should be tested without affecting live decisions.

TightenGate means the Gate should become stricter.

LoosenGate means the Gate should become less restrictive.

SplitGate means one Gate is too broad and should be divided by domain, risk, tool, or user class.

EscalateToHuman means the revision decision exceeds agent authority.

FreezeProtocol means the system should stop adapting temporarily because instability is detected.

Rollback means a prior Gate revision has failed.

This is important because not all Gate residual should produce the same response.

A FalseOpen pattern usually suggests tightening.

A FalseClose pattern may suggest loosening.

A FalseEscalate pattern may suggest better classification.

A FalseSilent pattern may suggest stronger risk detection.

A mixed pattern may suggest that one Gate is trying to govern too many contexts.

In that case, the correct revision is not simply stricter or looser. It is decomposition.

(4.7) SplitGateNeeded ⇔ FalseOpen and FalseClose both rise under the same Gate across distinct context classes.

For example, a general “legal advice” Gate may be too strict for summarizing a public legal document but too loose for advising a user about litigation strategy. The solution is not one threshold. The solution is separate Gates for separate commitment classes.

This gives an important design rule:

(4.8) When Gate errors conflict, split the Gate before tuning the threshold.

Meta-Gate also requires an evidence standard.

A Gate revision should not be accepted merely because the model proposes it. It should be attached to a RevisionRecord.

At minimum:

(4.9) GateRevisionRecord = (Trigger, OldGate, ProposedGate, SupportingTrace, ResidualClass,

RiskAssessment, TestPlan, RollbackRule).

This record prevents silent self-modification.

The agent should not be able to say:

I changed the Gate because it seemed better.

It should say:

I changed Gate G because trace set T showed repeated FalseClose in context C, with low irreversibility and confirmed human override. The new Gate applies only to context C. Shadow test S passed. Rollback triggers if FalseOpen increases beyond threshold θ.

This is accountable revision.

A Meta-Gate must also preserve the old Gate.

This is essential.

If a system overwrites the old Gate without preserving its trace, it loses the ability to explain why behavior changed.

Therefore:

(4.10) No Gate revision without OldGate preservation.

More generally:

(4.11) GateRevisionAllowed ⇔ TracePreserving ∧ ResidualHonest ∧ RollbackAvailable ∧ Testable ∧ AuthorityValid.

Each condition matters.

TracePreserving means the old behavior and reason for change remain auditable.

ResidualHonest means unresolved risk is not hidden by the revision.

RollbackAvailable means the system can return to the previous rule or safe baseline.

Testable means the change has observable success criteria.

AuthorityValid means the agent has permission to make this kind of change.

If any condition fails, revision should be blocked or escalated.

This is how self-correction avoids becoming self-drift.

It also connects to the deeper idea of autonomy.

An autonomous agent is not one that changes freely.

It is one that maintains coherent operation under changing conditions.

Thus:

(4.12) UngovernedSelfModification ≠ Autonomy.

(4.13) GovernedProtocolRevision = one condition of mature autonomy.

The Meta-Gate is therefore not a luxury module.

It is the safety condition for self-correcting AI.

Without Meta-Gate, revision becomes a new source of residual.

With Meta-Gate, revision becomes an auditable path from failure to improved protocol.

But Gate revision is only one part of self-correction.

The system must also discover what residual categories its original design failed to include.

That is the role of Residual Mining.

---

5. Residual Mining: How Agents Learn What Their Original Residual Taxonomy Missed

A residual taxonomy is a map of unresolved issues.

It tells the agent what kinds of incompleteness matter.

A simple residual taxonomy may include:

Missing evidence.

Ambiguous instruction.

Conflicting sources.

Tool failure.

Insufficient authority.

High-risk domain.

Unverified assumption.

Outdated information.

Unclear user intention.

Unsafe action.

These categories are useful.

But they are never complete.

A new domain may expose new residual.

A new tool may create new failure modes.

A new user group may interpret outputs differently.

A new document type may break old evidence rules.

A new policy may change what counts as safe.

A new attack pattern may exploit an old assumption.

Therefore residual categories must be mined from practice.

They cannot be fully specified in advance.

This gives:

(5.1) ResidualMining := process by which unresolved issues, repeated failures, and friction events are converted into reusable residual classes.

Residual Mining is how an agent learns the blind spots of its own residual taxonomy.

The process begins with experience.

But experience is not mere memory. It is trace that changes future projection, Gate, action, or revision.

Therefore Residual Mining must convert events into future constraints.

A minimal pipeline is:

(5.2) Application → FailureOrFriction → ResidualRecord → ResidualClass → GateUpdate or RevisionRuleUpdate.

Each step is necessary.

Application exposes the protocol to real conditions.

FailureOrFriction marks a mismatch between protocol and world.

ResidualRecord preserves the unresolved issue.

ResidualClass generalizes it beyond a single case.

GateUpdate or RevisionRuleUpdate makes the residual operational.

Without the final step, residual remains commentary.

A mature residual is not merely something the system says.

It is something that changes future behavior.

In compact form:

(5.3) StrongResidual ⇔ ∂FutureGate/∂Residual ≠ 0 or ∂FutureProjection/∂Residual ≠ 0.

This distinguishes real residual governance from vague caution.

A weak residual says:

There may be limitations.

A strong residual says:

The evidence is missing in source class S; do not give final answer until source class S is checked, or disclose the limitation and mark the answer tentative.

A weak residual says:

This may be uncertain.

A strong residual says:

This claim depends on assumption A. If A is false, conclusion C must be revised. Carry A forward as a revision trigger.

A weak residual says:

The tool failed.

A strong residual says:

Tool T timed out. Do not infer absence of data. Retry once under policy P. If still failed, escalate or answer with tool-failure residual.

Thus:

(5.4) WeakResidual = vague statement of uncertainty.

(5.5) StrongResidual = uncertainty with future operational consequence.

A useful ResidualRecord should therefore contain more than description.

At minimum:

(5.6) ResidualRecord = (UnresolvedIssue, Source, RiskLevel, AffectedGate, AffectedRevisionRule, EvidenceNeeded, EscalationPath, RevisionTrigger).

UnresolvedIssue describes what remains open.

Source identifies where the residual came from.

RiskLevel estimates consequence.

AffectedGate identifies which commitment rule is constrained.

AffectedRevisionRule identifies what kind of future change may be needed.

EvidenceNeeded specifies what would reduce the residual.

EscalationPath specifies who or what can resolve it.

RevisionTrigger specifies when it must be revisited.

This structure prevents residual from becoming decorative.

For example, in a technical document RAG agent, a ResidualRecord may be:

UnresolvedIssue: conflicting version of installation guide.

Source: document A dated 2024, document B dated 2026.

RiskLevel: medium.

AffectedGate: final technical instruction.

AffectedRevisionRule: retrieval ranking and version authority.

EvidenceNeeded: official latest release note.

EscalationPath: ask document owner or search authoritative repository.

RevisionTrigger: when latest version source is confirmed.

This residual is actionable.

It can constrain future answer, retrieval, and revision.

Now consider a prompt used for data extraction.

The prompt works well for ordinary invoices but fails on credit notes. At first this looks like an extraction error. Residual Mining should convert it into a new residual class:

ResidualClass: document-type polarity mismatch.

Meaning: source document reverses signs or economic direction relative to normal invoice.

Future consequence: Gate must require document type classification before extraction; schema must include sign convention; examples must include invoice and credit note.

This is how residual becomes protocol improvement.

Residual Mining has several sources.

User corrections are one source.

When users repeatedly correct the same kind of answer, the system should ask whether a new residual class is needed.

Tool failures are another source.

When a tool repeatedly fails under specific input conditions, the system should not treat each failure as isolated.

Ambiguous requirements are another source.

When users repeatedly clarify after the same instruction type, the declaration protocol may be underspecified.

Conflicting sources are another source.

When retrieval repeatedly finds contradictions, the evidence hierarchy may need revision.

Near misses are another source.

When an unsafe action almost happens but is caught by human review, the Gate has exposed residual.

Repeated output format errors are another source.

When schema failures repeat, projection instructions may lack strong attractor structure.

Over-caution is another source.

When the agent repeatedly refuses safe requests, the residual taxonomy may be too broad or the Gate too strict.

Thus:

(5.7) ResidualSources = {UserCorrection, ToolFailure, Ambiguity, SourceConflict, NearMiss, FormatFailure, OverCaution, HumanOverride, AuditFinding}.

Each source should feed the Residual Miner.

However, not every residual should become a new class.

A mature system must avoid taxonomy inflation.

If every unusual event becomes a new residual category, the system becomes bloated and unusable.

Residual Mining must therefore generalize carefully.

A new residual class should be added only when at least one of the following conditions holds:

The failure repeats across comparable contexts.

The failure has high risk.

The failure reveals a missing Gate condition.

The failure affects future projection.

The failure requires a new evidence rule.

The failure exposes a tool-body boundary.

The failure cannot be handled by existing residual classes.

This can be written as:

(5.8) AddResidualClass ⇔ Repetition ∨ HighRisk ∨ MissingGate ∨ FutureProjectionEffect ∨ NewEvidenceRule ∨ ToolBoundaryGap ∨ ExistingClassFailure.

This prevents residual taxonomy from becoming noise.

Residual Mining should also identify obsolete residual classes.

Some residual categories may become unnecessary after better tools, better retrieval, clearer policy, or stronger prompts are added.

A self-correcting system should not only add residual classes.

It should also prune, merge, or downgrade them.

(5.9) ResidualTaxonomyRevision = Add ∨ Split ∨ Merge ∨ Downgrade ∨ Retire.

Add means a new residual class is needed.

Split means one broad class hides distinct failure modes.

Merge means multiple classes are redundant.

Downgrade means the residual is lower risk than previously assumed.

Retire means the residual is no longer relevant under the current protocol.

For example, “missing citation” may be a major residual before citation tooling is added. After

the agent gains reliable source-linking and citation verification, some instances may be downgraded. But if citation failure still occurs in legal or technical answers, the residual may remain high priority.

This shows that residual categories are living protocol objects.

They are not static labels.

They are part of the agent's self-maintenance.

In compact form:

(5.10) ResidualTaxonomy is a revision-bearing object.

This is the bridge between residual and revision.

Once residual has been mined, classified, and tested, the system must decide whether to revise its protocol.

That leads to the next section.

---

6. Admissible Revision: Updating Without Erasing Accountability

Revision is necessary.

Without revision, a system cannot learn from residual.

Without revision, Gate failures repeat.

Without revision, projection instability persists.

Without revision, residual taxonomies become obsolete.

Without revision, tool-body maps fail to track real capabilities.

Without revision, the agent becomes brittle.

But revision is dangerous.

A system that changes its own protocol can damage itself.

It can overfit to a single case.

It can weaken safety constraints.

It can drift away from the declared task.

It can erase past trace.

It can suppress residual to appear more capable.

It can create circular self-justification.

It can oscillate between rules.

It can revise so often that no stable behavior remains.

Therefore revision must be admissible.

Admissible Revision is not free self-modification.

It is governed protocol evolution.

(6.1) AdmissibleRevision := revision that changes future protocol while preserving past trace, residual honesty, rollback possibility, testability, and declared boundary.

Each part of this definition is essential.

Revision changes future protocol.

If nothing changes future behavior, there has been no real revision.

Revision preserves past trace.

If the system erases or overwrites the past, it destroys accountability.

Revision preserves residual honesty.

If the system hides unresolved issues to simplify future operation, it becomes less trustworthy.

Revision preserves rollback possibility where possible.

If the revision fails, the system should be able to return to a safer previous state.

Revision is testable.

If no success condition exists, the revision is not engineering. It is drift.

Revision respects declared boundary.

If the system changes beyond its authority, it violates governance.

The simplest formula is:

(6.2) RevisionAllowed ⇔ TracePreserved ∧ ResidualPreserved ∧ Testable ∧ AuthorityValid ∧ RollbackDefined.

This is a necessary condition, not always sufficient. Some domains may require human approval, regulatory review, simulation, or formal verification.

A revision should also have positive expected value.

(6.3) RevisionGain = ErrorReduction − DriftCost − TraceDamage − ResidualSuppression − SafetyLoss.

Then:

(6.4) RevisionAllowed ⇔ RevisionGain > 0 ∧ TracePreserved ∧ ResidualPreserved ∧ TestPassed.

This formula is not a final mathematical law. It is a design discipline.

It forces the system to treat revision as a trade-off.

A revision that reduces one error but increases drift may not be acceptable.

A revision that improves user satisfaction but weakens evidence gates may not be acceptable.

A revision that reduces refusal rate but increases unsafe action may not be acceptable.

A revision that improves short-term performance but damages auditability may not be acceptable.

Every revision should therefore produce a RevisionRecord.

(6.5) RevisionRecord = (Trigger, OldRule, NewRule, SupportingTrace, ExpectedEffect, Risk, TestCase, RollbackRule).

Trigger explains why revision was considered.

OldRule preserves the previous protocol.

NewRule states the proposed change.

SupportingTrace shows the evidence.

ExpectedEffect states what improvement is expected.

Risk states what may get worse.

TestCase defines how the revision will be evaluated.

RollbackRule defines when to revert.

A mature agent should not simply say:

Updated policy.

It should say:

Revision R was triggered by residual class X appearing in 12 comparable cases. Old rule G blocked all requests of type A. Human review allowed 10 of 12 cases. New rule G′ permits A when evidence condition E and reversibility condition V hold. Shadow test passed. Rollback if FalseOpen exceeds threshold θ.

This is the difference between adaptation and admissible revision.

Revision has several common failure modes.

The first is policy drift.

Policy drift occurs when repeated small revisions move the system away from its declared purpose.

(6.6) PolicyDrift = distance(CurrentProtocol, DeclaredPurpose) increasing over revision sequence.

A system may drift because it overfits to user preferences, local metrics, or short-term success.

For example, a helpful assistant may gradually become too agreeable if user satisfaction is over-weighted. A code agent may gradually become too aggressive if task completion is over-weighted. A compliance agent may become too restrictive if risk avoidance is over-weighted.

The second failure mode is revision loop.

(6.7) RevisionLoop = Revision_n causes Residual_{n+1} that triggers Revision_{n+2} without convergence.

A system tightens a Gate, then false-close rises. It loosens the Gate, then false-open rises. It tightens again. The system oscillates.

This indicates that the Gate may need splitting, not tuning.

The third failure mode is trace erasure.

Trace erasure occurs when the system overwrites the past to make the new protocol appear natural.

This is especially dangerous.

A system that revises without preserving old trace cannot explain its change. It cannot compare old and new behavior. It cannot rollback reliably. It cannot audit.

Therefore:

(6.8) TraceErasure invalidates revision.

The fourth failure mode is residual suppression.

Residual suppression occurs when a revision reduces visible residual by hiding it rather than resolving it.

For example, a system may stop mentioning uncertainty to appear more confident. It may stop recording tool errors to reduce incident count. It may collapse ambiguous cases into default answers to improve completion rate.

This is false maturity.

(6.9) ResidualSuppression = apparent performance gain by reducing residual visibility rather than residual reality.

Residual suppression must be treated as a serious failure.

The fifth failure mode is overfitting to one case.

A dramatic failure may cause a system to revise too broadly.

For example, one bad tool output leads to disabling an entire tool family. One user complaint leads to changing all answer style. One legal ambiguity leads to refusing all legal document summaries.

A mature system should distinguish local residual from general protocol failure.

(6.10) LocalFailure does not imply GlobalRevision.

The sixth failure mode is self-justification.

A self-justifying agent revises its criteria so that its previous action appears correct.

This is especially dangerous in autonomous systems.

To prevent it, revision must be based on external trace, test cases, and where appropriate human or independent review.

(6.11) Revision must not be evaluated only by the same projection that proposed it.

This connects to cross-observer agreement.

Where risk is high, revision should require independent evaluators or redundant checks.

In practical terms:

(6.12) HighRiskRevision requires IndependentCheck.

The seventh failure mode is revision without authority.

An agent may detect a real problem but lack permission to change the relevant rule.

For example, a legal compliance Gate may be controlled by policy officers. A medical triage Gate may require clinical governance. A finance approval Gate may require senior authorization.

In such cases, the agent may propose revision but not execute it.

(6.13) RevisionAuthority ∈ {AgentAllowed, HumanApprovalRequired, PolicyOwnerRequired, Forbidden}.

This prevents the agent from exceeding its operational body.

Putting these together, an admissible revision process should follow this path:

(6.14) ResidualDetected → RevisionCandidate → MetaGateReview → Test → Approval → Deployment → Monitoring → RollbackIfNeeded.

Each step should leave trace.

No silent revision.

No untested revision.

No revision without residual record.

No revision without old rule preservation.

No revision without rollback where rollback is possible.

No revision that suppresses residual.

No revision outside authority.

These rules may sound strict, but they are what make self-correction safe.

They also make self-correction useful.

A system that cannot revise is brittle.

A system that revises freely is unstable.

A system that revises admissibly can mature.

This gives:

(6.15) BrittleAgent = NoRevision.

(6.16) UnstableAgent = UngovernedRevision.

(6.17) MatureAgent = AdmissibleRevision under MetaGate and Trace.

This completes the first half of the article’s second-order architecture.

We have shown:

Gate produces residual.

Meta-Gate governs Gate revision.

Residual Mining discovers missing residual classes.

Admissible Revision changes the protocol without erasing accountability.

The next problem is Projection.

Even if Gate, Residual, and Revision are governed, the agent still depends on language-based projection. The prompt or instruction may fail to produce a stable field collapse.

That is the subject of the next section.

7. Projection Instability: Why Language Cannot Be Fully Controlled by Declaration Alone

Gate, Residual, and Revision can be governed.

But one more problem remains.

Before an agent can gate commitment, it must first project something.

Projection is the act by which a larger field becomes visible under a declared protocol. A task, document, environment, user intention, legal issue, codebase, dataset, or tool result is never perceived in total. It is made readable through an observer frame.

For an LLM agent, projection is mediated by language.

The agent reads a prompt.

It receives context.

It interprets examples.

It follows system instructions.

It attends to retrieved documents.

It calls tools.

It forms an answer shape.

It decides what matters.

This entire process is projection.

In compact form:

(7.1) Projection = Prompt × Context × ModelPrior × Examples × ToolBody × OutputExpectation.

Projection is therefore not a purely mechanical transformation.

It is a semantic collapse process.

Even with the same prompt, different runs may emphasize different assumptions. Different models may project different answer structures. Different examples may pull the output into different basins. Additional context may shift attention. Ambiguous instructions may create multiple possible closures.

This is why projection instability is not merely a bug.

It is a natural feature of language-based world-making.

A prompt does not simply command an output. It creates an attractor field.

Sometimes the attractor is strong.

The agent repeatedly collapses similar inputs into a stable structure.

Sometimes the attractor is weak.

The agent drifts between formats, assumptions, tones, evidence standards, and closure styles.

Sometimes the attractor is conflicted.

The prompt pulls the model toward incompatible goals.

Sometimes the attractor is deceptive.

The prompt looks clear to the human designer but leaves critical operational gaps.

Therefore:

(7.2) Prompt ≠ DeterministicInstruction.

(7.3) Prompt = AttractorField for Projection.

This changes how we should think about prompt design.

A weak prompt says:

Analyze this deeply and give a useful answer.

This may produce good output in some cases, but the projection basin is broad. The model may write a philosophical essay, a practical checklist, a summary, a warning, a recommendation, or a speculative argument. If the user expects one of these and the model chooses another, projection failure occurs.

A stronger prompt declares:

Role.

Boundary.

Evidence rule.

Output shape.

Uncertainty handling.

Residual rule.

Gate condition.

Revision behavior.

This narrows the attractor basin.

The model is not merely told what to do. It is given a projection geometry.

Projection instability rises when the instruction leaves too much of this geometry undefined.

(7.4) ProjectionInstability ↑ when Boundary is vague, EvidenceRule is missing, Schema is weak, Goals conflict, or ResidualRule is absent.

Several common instability sources can be identified.

The first is boundary ambiguity.

The agent does not know what belongs inside the task-world.

A user asks:

Review this design.

Does this mean technical correctness, business feasibility, legal risk, UX clarity, cost, security, or all of them?

If the boundary is not declared, the agent invents a boundary.

The second is evidence ambiguity.

The agent does not know what counts as support.

Should it use the uploaded file only?

May it use general knowledge?

Must it search the web?

Should it cite sources?

Should it treat user-provided claims as assumptions or facts?

If the evidence rule is absent, the agent may over-project.

The third is output-shape ambiguity.

The agent does not know what closure form is expected.

Should it produce a table, essay, code, decision, plan, critique, short answer, or detailed analysis?

If output shape is undefined, repeated runs may collapse into different forms.

The fourth is goal conflict.

The prompt may ask the agent to be concise, exhaustive, creative, cautious, persuasive, rigorous, and practical at the same time. These goals are not always compatible.

The model then chooses a compromise basin, often without declaring what it sacrificed.

The fifth is authority ambiguity.

The agent does not know whether it is allowed to decide, recommend, draft, execute, or only analyze.

This is especially dangerous in tool-use systems.

The sixth is residual silence.

The prompt does not instruct the agent to preserve unresolved issues.

The model then treats fluency as closure.

The seventh is revision silence.

The prompt does not specify what to do after contradiction, failure, user correction, or tool error.

The model may apologize, overcorrect, defend itself, or drift.

The eighth is style dominance.

The prompt heavily emphasizes tone, persona, persuasion, or elegance, while evidence and boundary rules remain weak.

The projection may then collapse into rhetorical quality rather than truth-preserving structure.

This is common in long-form writing agents.

The ninth is context overload without hierarchy.

The agent receives many documents, rules, constraints, and examples, but no ordering principle.

Attention spreads.

Projection becomes unstable.

The tenth is hidden tool-body uncertainty.

The agent is told to use tools, but the tool boundaries, failure modes, costs, and rollback paths are not declared.

The tool becomes an unstable extension of the projection field.

These sources can be summarized:

(7.5) ProjectionFragility = Ambiguity + GoalConflict + Underconstraint + ContextDrift + ToolUncertainty + ResidualSilence.

This is why prompt stability must be treated as an engineering object, not an artistic instinct.

Human civilization already discovered this problem long before AI.

A legal contract is a projection-stabilizing device.

It does not merely express intention. It narrows future interpretation.

An accounting standard is a projection-stabilizing device.

It tells accountants how to see transactions.

A factory SOP is a projection-stabilizing device.

It turns ambiguous work into repeatable action.

A medical guideline is a projection-stabilizing device.

It shapes diagnosis, caution, evidence, and escalation.

A military command format is a projection-stabilizing device.

It reduces ambiguity under pressure.

A school examination rubric is a projection-stabilizing device.

It tells both student and marker what counts as valid performance.

An advertising copy formula is a projection-stabilizing device.

It repeatedly pulls audience attention into predictable affective and cognitive basins.

A judicial precedent is a projection-stabilizing device.

It guides future interpretation by preserving past collapse.

Human societies do not rely on raw instruction alone. They build forms.

Forms stabilize projection.

Templates stabilize projection.

Examples stabilize projection.

Rituals stabilize projection.

Training stabilizes projection.

Repeated practice stabilizes projection.

This suggests an important claim for AI:

(7.6) ProjectionGovernance ≠ making language deterministic.

(7.7) ProjectionGovernance = shaping the collapse basin.

The aim is not to remove all variation. Some variation is useful. Creativity, adaptation, and sensitivity to context require flexible projection.

The aim is to prevent destructive variation.

A mature agent should vary within a declared basin, not drift across incompatible basins.

Thus:

(7.8) GoodProjectionVariance = variation inside declared output basin.

(7.9) BadProjectionVariance = drift across incompatible basins.

For example, in legal summarization, wording may vary, but jurisdiction, source citation, uncertainty, and limitation handling should remain stable.

In coding, implementation details may vary, but test logic, file boundaries, and rollback awareness should remain stable.

In customer support, tone may vary, but policy boundary and escalation rules should remain stable.

In research, interpretation may vary, but evidence status and residual disclosure should remain stable.

This is the foundation of Strong-Attractor Projection.

---

8. Strong-Attractor Projection: From Prompt Writing to Prompt Governance

A strong prompt is not merely a clear prompt.

A strong prompt is an attractor-forming instruction.

It repeatedly collapses comparable tasks into a stable projection basin.

This gives the core definition:

(8.1) StrongAttractorInstruction := instruction structure that repeatedly collapses comparable agents, contexts, and runs into a stable projection basin.

Strong-Attractor Projection is the runtime process produced by such instruction.

(8.2) StrongAttractorProjection = StableCollapse(Prompt, ContextClass, ModelClass, OutputBasin).

The phrase “comparable” matters.

No prompt is stable across all possible worlds.

A prompt that works for accounting document extraction may fail for poetry.

A prompt that works for legal memo drafting may fail for engineering debugging.

A prompt that works for one model may fail for another.

A prompt that works with complete evidence may fail under missing evidence.

A prompt that works in a reversible drafting environment may fail when tool actions become irreversible.

Therefore prompt stability must always be scoped.

(8.3) Stability is defined over ContextClass and ModelClass, not over all possible inputs.

A StrongAttractorInstruction has several components.

The first is role declaration.

The agent must know what kind of observer it is.

A reviewer sees differently from a coder.

A judge sees differently from an advocate.

A teacher sees differently from an examiner.

A safety auditor sees differently from a product designer.

A researcher sees differently from a marketer.

Role declaration fixes observer frame.

(8.4) RoleDeclaration stabilizes ObserverFrame.

The second is task boundary.

The agent must know what belongs inside the task and what remains outside.

Boundary prevents uncontrolled expansion.

(8.5) TaskBoundary stabilizes WorldScope.

The third is evidence rule.

The agent must know what counts as support.

Uploaded files only?

Current web sources?

User-provided assumptions?

Internal memory?

Tool outputs?

Direct calculation?

Expert judgment?

Evidence rule prevents hallucinated grounding.

(8.6) EvidenceRule stabilizes Grounding.

The fourth is output schema.

The agent must know the expected closure shape.

Essay.

Table.

JSON.

Checklist.

Decision memo.

Diagnostic report.

Step-by-step plan.

Code patch.

Output schema stabilizes collapse form.

(8.7) OutputSchema stabilizes CollapseShape.

The fifth is example structure.

Positive examples show the basin.

Negative examples mark forbidden basins.

Without examples, the model may still infer the desired structure, but the attractor is weaker.

(8.8) Examples define BasinCenter.

(8.9) NegativeExamples define BasinBoundary.

The sixth is residual rule.

The agent must know what unresolved issues should be preserved.

Missing evidence.

Ambiguous requirement.

Conflicting source.

Tool failure.

Risk condition.

Assumption dependence.

Without a residual rule, the model may seek fluent closure.

(8.10) ResidualRule prevents FalseClosure.

The seventh is Gate rule.

The agent must know when not to answer, act, or finalize.

It may need to ask clarification.

It may need to search.

It may need to escalate.

It may need to produce a draft rather than final advice.

It may need to refuse a dangerous action.

(8.11) GateRule prevents PrematureCommitment.

The eighth is revision rule.

The agent must know what to do after correction, contradiction, missing evidence, or failed tool action.

Without revision rule, the agent may drift.

(8.12) RevisionRule stabilizes PostFailureBehavior.

The ninth is self-audit rubric.

The agent should be able to evaluate its own output against declared criteria.

This is not enough by itself, but it strengthens the attractor.

(8.13) SelfAuditRubric stabilizes InternalReview.

Putting these together:

(8.14) StrongAttractorInstruction = Role + Boundary + EvidenceRule + OutputSchema + Examples + NegativeExamples + ResidualRule + GateRule + RevisionRule + SelfAuditRubric.

This formula is not a requirement that every prompt must be long.

In low-risk, simple tasks, many components can be implicit.

In high-risk, long-horizon, tool-using, or domain-specific tasks, they should be explicit.

Thus:

(8.15) PromptStructureNeeded ↑ as Risk × Ambiguity × Irreversibility × Horizon ↑.

A strong prompt is not necessarily long.

A long prompt can be weak if it lacks hierarchy.

A short prompt can be strong if it activates a well-tested shared template.

For example:

“Summarize this meeting in the standard action-item format.”

This may be strong inside an organization where “standard action-item format” is already a trace-proven attractor.

But for a new model or outside that organization, the same prompt may be weak.

Therefore:

(8.16) StrongPrompt ≠ LongPrompt.

(8.17) StrongPrompt = AttractorFormingInstruction + ResidualGovernance + GateCompatibility.

Strong-Attractor Projection also requires compatibility with Gate and Residual.

A prompt may produce beautiful output but hide uncertainty.

That is not mature.

A prompt may produce stable format but permit unsafe action.

That is not mature.

A prompt may produce persuasive explanations but weaken evidence standards.

That is not mature.

A prompt may produce confident answers but suppress residual.

That is not mature.

Therefore a strong attractor must be aligned with the runtime's Gate and Residual policies.

(8.18) StrongAttractorValidity = ProjectionStability + GateCompatibility + ResidualHonesty.

This is why prompt governance must be connected to agent governance.

A prompt cannot be evaluated only by output quality.

It must be evaluated by the kind of world it repeatedly makes visible.

Does it make evidence visible?

Does it make uncertainty visible?

Does it make assumptions visible?

Does it make risk visible?

Does it make revision triggers visible?

Does it make authority boundaries visible?

Does it make tool consequences visible?

If not, it may be a strong rhetorical attractor but a weak operational attractor.

This distinction is crucial.

(8.19) RhetoricalAttractor = stable fluency without reliable governance.

(8.20) OperationalAttractor = stable projection with trace, residual, gate, and revision compatibility.

Modern LLMs are very good at rhetorical attractors.

They can produce fluent essays, summaries, plans, and explanations.

But mature Enactive AI requires operational attractors.

The projection must support future action, audit, residual preservation, and controlled revision.

This leads to a new object: the Projection Stability Claim.

A prompt should not simply be called stable or unstable.

Its stability must be stated with evidence status.

That is the subject of the next section.

---

9. Stability Claims: Trace-Proven and Structure-Inferred Stability

Projection stability is not a binary property of a prompt.

It is a claim.

And every claim has an evidence status.

A prompt may be stable because it has already been tested many times.

Or it may only appear stable because its structure resembles known stabilizing forms.

These are different.

They should not be confused.

This section introduces two kinds of stability:

(9.1) TraceProvenStability := stability supported by repeated historical traces across comparable contexts.

(9.2) StructureInferredStability := stability inferred from known attractor-forming structures before sufficient historical validation exists.

The first is empirical.

The second is inferential.

Trace-Proven Stability means:

This prompt pattern is stable because it has survived use.

It has been run repeatedly.

It has been tested across comparable cases.

Its failure modes are known.

Its output variance is low.

Its residual behavior is predictable.

Its Gate compatibility has been observed.

Its performance is supported by trace.

Examples include:

A JSON extraction prompt tested on hundreds of documents.

A legal memo template repeatedly reviewed by lawyers.

A coding diagnostic prompt that consistently separates cause, evidence, fix, and risk.

A customer-service script validated by A/B testing.

A factory SOP that has a long record of low incident rate.

A financial report checklist that has survived repeated audit cycles.

A technical documentation template that reduces support errors.

These are trace-proven attractors.

Their strength does not come only from clever design.

It comes from survival under use.

In compact form:

(9.3) TraceProvenStability(P) = RepeatedSuccessfulCollapse(P, ContextClass, ModelClass, OutputBasin).

But trace-proven does not mean universally valid.

A prompt tested on simple invoices may fail on credit notes.

A legal memo prompt tested in one jurisdiction may fail in another.

A coding prompt tested on Python may fail on VBA.

A customer-service script tested in one culture may fail in another.

Trace only proves stability inside a context class.

Therefore:

(9.4) TraceProvenStability is bounded by ContextSimilarity.

If context shifts, trace support weakens.

This is why production systems need domain shift detection.

Structure-Inferred Stability is different.

It means:

This prompt is likely stable because its structure resembles known attractor-forming forms.

It may not yet have extensive historical trace.

But it contains stabilizing elements.

It declares role.

It defines boundary.

It specifies evidence.

It gives output schema.

It includes examples.

It defines residual handling.

It defines Gate behavior.

It defines revision behavior.

It includes self-audit.

Therefore expert judgment estimates that it will collapse into a stable basin.

This is an engineering hypothesis.

Not a proven fact.

In compact form:

(9.5) StructureInferredStability(P) = ExpectedStableCollapse based on AttractorFormingStructure(P).

This kind of stability is necessary.

New tasks cannot always wait for long historical validation.

A new domain, new tool, new workflow, or new document type may require prompt design before production trace exists.

In such cases, expert structure matters.

But Structure-Inferred Stability must be labeled as such.

It carries residual.

(9.6) StructureInferredStability carries ValidationResidual.

The agent or designer should not say:

This prompt is stable.

It should say:

This prompt is expected to be stable because it contains boundary, schema, evidence rule, residual rule, and Gate rule, but it has not yet been trace-proven under production conditions.

This is a different epistemic status.

Many practical prompts are hybrid.

They have some trace support and some structure-based inference.

For example, a document extraction prompt may be trace-proven for invoices but only structure-inferred for purchase orders. A legal analysis prompt may be trace-proven for summarizing cases but only structure-inferred for drafting arguments. A coding agent prompt may be trace-proven for syntax fixes but only structure-inferred for architectural refactoring.

Thus:

(9.7) HybridStability = PartialTraceEvidence + StructuralInference.

This is the most common state in real systems.

The important point is to record the difference.

A Projection Stability Claim should therefore include:

(9.8) ProjectionStabilityClaim = (PromptPattern, TaskDomain, ExpectedProjectionBasin, StabilityBasis, EvidenceTrace, StructuralReasons, KnownInstabilityModes, Residual, RevisionTrigger).

StabilityBasis may take values such as:

Trace-Proven.

Structure-Inferred.

Hybrid.

Unknown.

Known-Unstable.

The claim should also specify the expected projection basin.

For example:

ExpectedProjectionBasin = “three-part technical answer: diagnosis, evidence, fix.”

ExpectedProjectionBasin = “JSON object following schema S.”

ExpectedProjectionBasin = “legal issue memo with facts, issue, rule, analysis, residual.”

ExpectedProjectionBasin = “coding plan with files affected, risks, test strategy, rollback.”

Without an expected basin, stability cannot be measured.

This gives:

(9.9) No StabilityClaim without ExpectedProjectionBasin.

A useful stability model can be written:

(9.10) StabilityClaim(P) = EvidenceSupport(P) + StructuralSupport(P) − InstabilityRisk(P).

EvidenceSupport is based on trace.

(9.11) EvidenceSupport(P) = f(repetition, context_similarity, model_coverage, failure_rate, variance).

Repetition measures how often the prompt has been run.

Context similarity measures how close new cases are to tested cases.

Model coverage measures whether the prompt works across models or only one.

Failure rate measures historical breakdown.

Variance measures output spread.

StructuralSupport is based on design.

(9.12) StructuralSupport(P) = f(boundary, schema, examples, evidence_rule, residual_rule, gate_rule).

Boundary defines scope.

Schema defines output shape.

Examples define basin center.

Evidence rule defines grounding.

Residual rule prevents false closure.

Gate rule prevents unsafe commitment.

InstabilityRisk is based on warning signs.

(9.13) InstabilityRisk(P) = g(ambiguity, conflict, underconstraint, context_drift, adversarial_pressure, tool_uncertainty).

Ambiguity creates multiple projections.

Conflict creates attractor competition.

Underconstraint leaves too much free collapse.

Context drift weakens trace support.

Adversarial pressure exploits instability.

Tool uncertainty expands hidden action risk.

Together:

(9.14) StrongAttractorStatus(P) requires StabilityClaim(P) above threshold and Residual explicitly recorded.

This prevents a common mistake.

A prompt should not be called a Strong Attractor merely because it is well-written.

It becomes a Strong Attractor only when its stability claim is either trace-proven or structurally well-supported, with residuals explicitly recorded.

This gives the central principle:

(9.15) Projection stability is not a property asserted by confidence; it is a claim governed by evidence status.

This principle is especially important for AI systems that design prompts for themselves.

If an agent writes a new instruction for a sub-agent, it should not simply assume the prompt is stable.

It should label the stability basis.

Trace-proven?

Structure-inferred?

Hybrid?

Unknown?

Known fragile?

This makes prompt governance part of runtime governance.

But stability is only one side.

We must also classify instability.

---

10. Instability Claims: Known Fragility and Predicted Fragility

Just as stability has evidence status, instability also has evidence status.

Some instability is known from prior failures.

Some instability is predicted from structure before enough testing exists.

These are different.

They should not be confused.

We define:

(10.1) TraceProvenInstability := instability demonstrated by repeated projection divergence under comparable runs.

(10.2) StructureInferredInstability := predicted projection fragility inferred from underconstraint, contradiction, attractor conflict, or missing Gate and Residual rules.

Trace-Proven Instability means the prompt or protocol has already failed repeatedly.

The evidence may include:

Output format breaks.

Contradictory answers across repeated runs.

High variance under the same prompt.

Model-specific collapse.

Failure under paraphrase.

Tool-use loops.

Citation errors.

Unsupported claims.

Repeated user corrections.

Repeated over-refusal.

Repeated unsafe passes.

Repeated revision loops.

In compact form:

(10.3) TraceProvenInstability(P) = RepeatedProjectionDivergence(P, ComparableContext).

This kind of instability is strong evidence.

The system should not keep using the prompt as if nothing is known.

A trace-proven fragile prompt should be repaired, restricted, shadow-tested, or retired.

(10.4) KnownFragilePattern should not remain in unrestricted production.

Structure-Inferred Instability is different.

The prompt may not yet have failed many times.

But its structure contains warning signs.

For example:

The task boundary is unclear.

The output schema is missing.

The prompt has conflicting goals.

The evidence rule is absent.

The residual rule is absent.

The Gate rule is absent.

The examples conflict.

The instruction is too broad.

The prompt emphasizes style more than truth.

The prompt asks for final answer despite missing critical information.

The tool action is not bounded.

The context is long but unranked.

The prompt mixes planning, execution, and evaluation without phase separation.

Even before testing, we can infer fragility.

(10.5) StructureInferredInstability(P) = FragilityPredicted from MissingStructure and AttractorConflict.

This is not proof of failure.

It is a warning.

The proper response is not always rejection. It may be repair before testing.

For example, if the output schema is missing, add one.

If evidence rule is absent, define it.

If residual rule is absent, add one.

If goals conflict, prioritize them.

If examples conflict, remove or separate them.

If tool action is unbounded, define tool-body constraints.

If context is long, add hierarchy.

If final answer is demanded under ambiguity, add clarification Gate.

Thus:

(10.6) SuspectedFragility should trigger StructureRepair before production use.

A mature agent should distinguish known fragility from suspected fragility.

Known fragility is empirical.

Suspected fragility is structural.

Both matter.

But they require different action.

(10.7) KnownFragility → restrict, repair, or retire.

(10.8) SuspectedFragility → repair, test, and classify.

This distinction allows AI systems to be proactive.

They do not need to wait for repeated failure before improving prompts.

Human experts often do this naturally.

A senior lawyer can look at a contract clause and predict ambiguity before litigation occurs.

An experienced accountant can see a reporting instruction and predict manipulation risk.

A factory supervisor can read an SOP and predict operator confusion.

A teacher can inspect exam wording and predict student misinterpretation.

A software architect can read an API specification and predict misuse.

They are not relying only on trace-proven failure.

They are using structure-inferred instability.

AI systems can be trained or instructed to do the same.

This leads to a broader principle:

(10.9) MaturePromptGovernance = TraceLearning + StructuralJudgment.

TraceLearning learns from what has failed.

StructuralJudgment predicts what may fail.

Together they form a better stability discipline.

However, both can be wrong.

A prompt may look fragile but work well because the model has strong prior training.

A prompt may look stable but fail under adversarial pressure.

A trace-proven prompt may fail after model upgrade.

A structure-inferred prompt may work in one domain and fail in another.

Therefore all stability and instability claims should carry residual.

(10.10) Every StabilityClaim carries DomainShiftResidual.

(10.11) Every InstabilityClaim carries FalseAlarmResidual.

DomainShiftResidual means the prompt may fail outside the tested or inferred context.

FalseAlarmResidual means a predicted instability may not actually materialize.

This is why Prompt Governance must be ledgered.

The system should record not only whether a prompt works, but why it was believed to work or fail.

This gives a practical object:

(10.12) InstabilityClaim = (PromptPattern, InstabilityBasis, FailureEvidence, StructuralWarningSigns, AffectedBasin, RiskLevel, RepairAction, RetestTrigger).

InstabilityBasis may be trace-proven, structure-inferred, hybrid, or unknown.

FailureEvidence stores historical breakdown.

StructuralWarningSigns store design concerns.

AffectedBasin identifies which output structure is unstable.

RiskLevel estimates consequence.

RepairAction proposes intervention.

RetestTrigger defines when to audit again.

This makes instability actionable.

Not all instability is bad.

Some prompts are deliberately open.

Creative brainstorming, philosophical exploration, speculative research, poetry, and concept generation may benefit from wide projection basins.

In such cases, instability is not automatically a defect.

The issue is whether the instability is declared and appropriate.

(10.13) OpenProjection is acceptable when Exploration is declared and CommitmentGate remains separate.

A brainstorming prompt may allow broad variation.

But it should not directly trigger irreversible action.

A speculative theory prompt may explore unusual analogies.

But it should not present them as verified facts.

A creative prompt may generate multiple possibilities.

But if the user asks for final decision, a Gate must later evaluate.

Thus:

(10.14) ExplorationVariance is allowed before Gate.

(10.15) CommitmentVariance must be controlled at Gate.

This distinction prevents over-stabilization.

Not every agentic process should be rigid.

The goal is not maximum determinism.

The goal is correct stability at the right phase.

This leads naturally to the Four-Quadrant Stability Map.

---

11. The Four-Quadrant Stability Map

Prompt and protocol stability can be understood through two axes.

The first axis is empirical evidence.

Is there strong historical trace showing how this prompt or protocol behaves?

The second axis is structural judgment.

Does the prompt or protocol look attractor-forming or fragile based on its design?

Together they form four quadrants.

(11.1) StabilityQuadrant = EvidenceAxis × StructureAxis.

The table is:

	Empirical evidence strong	**Empirical evidence weak**
Structure looks stable	Validated Strong Attractor	Promising Attractor Hypothesis
Structure looks unstable	Known Fragile Pattern	Suspected Fragile Pattern

Each quadrant has a different governance response.

11.1 Validated Strong Attractor

A Validated Strong Attractor has both structural strength and empirical support.

It looks stable by design.

It has also survived repeated use.

Its expected projection basin is known.

Its common residuals are known.

Its failure modes are known.

Its output variance is low under comparable context.

Its Gate compatibility has been tested.

This is the best candidate for production use.

(11.2) ValidatedStrongAttractor = HighStructuralSupport + HighEvidenceSupport + LowInstabilityRisk.

Examples:

A tested invoice extraction prompt with low schema failure.

A legal summary template repeatedly reviewed by experts.

A code review prompt with stable risk categories and known output structure.

A technical support answer format validated across thousands of tickets.

A workflow instruction that consistently produces auditable trace.

However, even Validated Strong Attractors require monitoring.

They may fail under domain shift, model upgrade, new tool integration, adversarial input, or policy change.

Therefore:

(11.3) ValidatedStrongAttractor still carries DomainShiftResidual.

Production use should include drift detection.

11.2 Promising Attractor Hypothesis

A Promising Attractor Hypothesis looks structurally strong but lacks enough empirical trace.

It has role, boundary, evidence rule, schema, examples, residual rule, Gate rule, and revision logic.

But it has not yet been tested enough.

This is suitable for pilot, shadow testing, limited rollout, or low-risk deployment.

(11.4) PromisingAttractorHypothesis = HighStructuralSupport + LowEvidenceSupport.

It should not be treated as production-proven.

Its residual should be explicit:

ValidationResidual remains.

The proper action is to test.

Run repetition tests.

Run perturbation tests.

Run model variation tests.

Run edge cases.

Run adversarial ambiguity.

Run human review.

Collect trace.

If successful, it may move toward Validated Strong Attractor.

(11.5) PromisingAttractor → Testing → ValidatedStrongAttractor or FragilePattern.

This is how new prompts mature.

11.3 Known Fragile Pattern

A Known Fragile Pattern has empirical evidence of instability, and its structure also appears weak.

It has failed repeatedly.

Its failure is explainable.

It may lack boundary, evidence rule, schema, residual rule, Gate rule, or hierarchy.

It may contain conflicting goals.

It may produce high output variance.

It may create false closure.

It may trigger tool loops.

This quadrant is dangerous.

(11.6) KnownFragilePattern = HighFailureEvidence + HighStructuralWarning.

Such prompts should be blacklisted, rewritten, restricted, or used only in exploration mode.

They should not remain in production commitment paths.

(11.7) KnownFragilePattern → Retire or Redesign.

If the prompt must still be used, the Gate should prevent direct commitment.

11.4 Suspected Fragile Pattern

A Suspected Fragile Pattern has weak empirical evidence but appears structurally unstable.

It has not yet failed enough times to be trace-proven fragile.

But experienced judgment predicts fragility.

This is common in new workflows.

Examples:

A broad agent instruction with no output schema.

A tool-use prompt that does not define rollback.

A research prompt that does not separate evidence from speculation.

A coding prompt that asks for changes without asking for tests.

A legal prompt that ignores jurisdiction.

A medical prompt that ignores missing patient context.

A long prompt with many constraints but no hierarchy.

(11.8) SuspectedFragilePattern = LowEvidenceSupport + HighStructuralWarning.

The proper response is repair before production.

Add boundary.

Add schema.

Add evidence rule.

Add residual rule.

Add Gate rule.

Add examples.

Add hierarchy.

Add revision behavior.

Then test.

(11.9) SuspectedFragilePattern → StructureRepair → StabilityAudit.

The Four-Quadrant Map gives a practical governance classification.

It prevents two common mistakes.

The first mistake is empirical arrogance.

A team says:

This prompt worked before, so it is stable.

But if context changes, trace support may not transfer.

The second mistake is structural arrogance.

A designer says:

This prompt is well-designed, so it is stable.

But without testing, this is only Structure-Inferred Stability.

The map forces both evidence and structure to be considered.

This gives a production readiness formula:

(11.10) ProductionReadiness(P) = StabilityQuadrant(P) × RiskLevel(Task) × Irreversibility(Action).

For low-risk reversible tasks, a Promising Attractor Hypothesis may be acceptable.

For high-risk irreversible tasks, only a Validated Strong Attractor with strong Gate and human oversight may be acceptable.

For exploratory thinking, a Suspected Fragile Pattern may be acceptable if no commitment is made.

For automated action, a Known Fragile Pattern should be blocked.

This is the core of prompt governance.

But governance requires measurement.

That leads to Projection Stability Audit.

---

12. Projection Stability Audit

Projection Stability Audit is the process of testing whether a prompt or instruction reliably collapses comparable contexts into the expected projection basin.

It should not rely on subjective impression alone.

A prompt may look good and fail.

A prompt may look ordinary and work because it activates a known pattern.

A prompt may work in one model but fail in another.

A prompt may work under clean context but fail under noisy context.

A prompt may work for ordinary cases but fail under edge cases.

Therefore stability must be audited.

We can define four levels.

12.1 Level 1 — Structural Audit

Structural Audit examines the prompt itself.

It asks:

Does the prompt declare the observer role?

Does it define task boundary?

Does it define evidence rule?

Does it define output shape?

Does it define residual rule?

Does it define Gate rule?

Does it define revision rule?

Does it include examples or counterexamples where needed?

Does it establish hierarchy among constraints?

Does it separate exploration from commitment?

A basic score can be written:

(12.1) StructuralAuditScore = score(Role, Boundary, Schema, EvidenceRule, ResidualRule, GateRule, RevisionRule, Examples, Hierarchy).

A high StructuralAuditScore supports Structure-Inferred Stability.

It does not prove stability.

It only strengthens the hypothesis.

12.2 Level 2 — Repetition Audit

Repetition Audit runs the same prompt across repeated trials under comparable conditions.

It asks:

Does the answer structure remain stable?

Does the evidence rule remain stable?

Does residual disclosure remain stable?

Does the Gate behavior remain stable?

Does the model repeatedly produce the expected basin?

A simple score:

(12.2) RepetitionStability = 1 – OutputVarianceAcrossRuns.

OutputVariance may be measured by schema pass rate, semantic similarity, structural similarity, field coverage, or human evaluation.

For structured tasks, schema pass rate may be enough.

For analytical writing, structural similarity may matter more.

For decision tasks, Gate consistency may be central.

Repetition Audit helps detect random drift.

But it is still not enough.

A prompt may be stable only under identical wording.

Therefore we need perturbation.

12.3 Level 3 — Perturbation Audit

Perturbation Audit tests whether the projection basin survives controlled variation.

It asks:

If the user paraphrases the task, does the output remain structurally stable?

If irrelevant context is added, does the model ignore it?

If a conflicting source appears, does the residual rule activate?

If a tool fails, does the agent avoid false inference?

If the model changes, does the projection remain similar?

If edge cases appear, does the Gate respond correctly?

If ambiguity increases, does the agent ask clarification rather than hallucinate?

A simple metric:

(12.3) PerturbationRobustness = StableRunsUnderPerturbation / TotalPerturbationRuns.

Perturbation types may include:

Paraphrase.

Context noise.

Missing information.

Conflicting evidence.

Tool failure.

Model variation.

Adversarial ambiguity.

Edge cases.

Long context.

Policy conflict.

A prompt that passes Repetition Audit but fails Perturbation Audit is fragile.

It is stable only in narrow conditions.

12.4 Level 4 — Deployment Audit

Deployment Audit observes the prompt under real use.

It asks:

Does the prompt remain stable with real users?

Does it survive real documents?

Does it handle real tool failures?

Does it preserve residual under time pressure?

Does it avoid overcommitment?

Does it reduce human correction?

Does it maintain Gate compatibility?

Does it generate useful trace?

A simple metric:

(12.4) DeploymentStability = 1 − ProductionFailureRate.

But production failure rate should be decomposed.

A useful audit should measure:

Schema failure.

Evidence failure.

Residual failure.

Gate failure.

Revision failure.

Tool failure.

User correction rate.

Human override rate.

Unsafe near miss.

Output variance.

Latency cost.

This gives a fuller deployment score:

(12.5) DeploymentFailure = SchemaFail + EvidenceFail + ResidualFail + GateFail + RevisionFail + ToolFail + HumanCorrection + UnsafeNearMiss.

A combined Projection Stability Score can be written:

(12.6) ProjectionStabilityScore = w_1 StructuralAuditScore + w_2 RepetitionStability + w_3 PerturbationRobustness + w_4 DeploymentStability.

The weights should depend on risk.

For a new low-risk prompt, StructuralAudit and Repetition may matter most.

For a production high-risk workflow, DeploymentStability and PerturbationRobustness should dominate.

(12.7) AuditWeight shifts toward empirical evidence as Risk and Irreversibility increase.

Projection Stability Audit should output a StabilityBasis.

Possible outputs:

Trace-Proven Stable.

Structure-Inferred Stable.

Hybrid Stable.

Suspected Fragile.

Known Fragile.

Unknown.

It should also output residual.

For example:

StabilityBasis: Structure-Inferred Stable.

Residual: No production trace yet; perturbation under conflicting evidence not tested.

RevisionTrigger: run 100 shadow cases; if residual disclosure fails above 5%, revise prompt.

Or:

StabilityBasis: Trace-Proven Stable.

Residual: Tested only on English documents; cross-lingual domain shift untested.

RevisionTrigger: if non-English failure rate exceeds threshold, split prompt by language.

Or:

StabilityBasis: Known Fragile.

Residual: Output schema fails under nested tables.

RevisionTrigger: add table-specific extraction phase and retest.

This turns prompt design into prompt governance.

The final audit object can be written:

(12.8) ProjectionAuditRecord = (PromptPattern, ExpectedBasin, StructuralScore, RepetitionScore, PerturbationScore, DeploymentScore, StabilityBasis, Residual, RevisionTrigger).

A mature agent should be able to carry this object.

When asked to use or generate a prompt, it should not only produce wording.

It should estimate stability status.

Is this prompt trace-proven?

Is it structure-inferred?

Is it hybrid?

Is it fragile?

What residual remains?

What test is needed?

This is especially important for agent-generated skills.

As AI agents begin to create their own prompts, workflows, tool instructions, and sub-agent roles, prompt governance becomes a safety issue.

An agent that writes unstable instructions for itself can create cascading projection failures.

Therefore:

(12.9) AgentGeneratedInstruction requires ProjectionAudit before high-risk use.

This leads into the next layer.

Prompt governance is not isolated.

It must connect to the whole agent runtime: Gate, Residual, Revision, Tool-Body, Trace, and Human Escalation.

That is the subject of the next section.

13. From Prompt Governance to Agent Governance

Prompt governance is necessary.

It is not sufficient.

A prompt is not an isolated object. It operates inside an agent body.

That body may include tools, memory, APIs, retrieval systems, file access, execution permissions, budgets, latency constraints, policy gates, trace ledgers, residual ledgers, rollback systems, and human escalation paths.

Therefore a stable prompt can still produce unsafe behavior if the surrounding agent runtime is weak.

A strong instruction may produce a good projection, but if the Gate is too loose, the agent may act too early.

A strong instruction may preserve residual, but if the ledger does not carry residual forward, future behavior will not change.

A strong instruction may specify caution, but if tool actions are not body-mapped, the agent may still create external side effects.

A strong instruction may produce a good plan, but if revision is ungoverned, later correction may drift.

Thus:

(13.1) PromptGovernance ⊂ AgentGovernance.

Agent governance includes prompt governance, but extends beyond it.

A mature agent must govern the full path:

(13.2) Projection → Gate → Action → Trace → Residual → Revision.

Prompt governance mainly stabilizes Projection.

Agent governance stabilizes the entire loop.

This gives:

(13.3) AgentGovernance = ProjectionGovernance + GateGovernance + ResidualGovernance + RevisionGovernance + ToolBodyGovernance + TraceGovernance.

Each layer has a distinct question.

ProjectionGovernance asks:

What world does the prompt make visible?

GateGovernance asks:

When may the agent commit?

ResidualGovernance asks:

What remains unresolved and how does it constrain the future?

RevisionGovernance asks:

When may the protocol change?

ToolBodyGovernance asks:

What can the agent perceive, change, damage, or recover?

TraceGovernance asks:

What must be preserved so that the system remains accountable?

These layers must be aligned.

If ProjectionGovernance is strong but GateGovernance is weak, the system may produce high-quality but unsafe action.

If GateGovernance is strong but ProjectionGovernance is weak, the system may block too much or misclassify risk.

If ResidualGovernance is strong but RevisionGovernance is weak, the system accumulates unresolved issues without learning.

If RevisionGovernance is strong but TraceGovernance is weak, the system changes without accountability.

If ToolBodyGovernance is weak, even good reasoning may produce bad effects.

Therefore:

(13.4) LocalStability does not imply RuntimeStability.

A stable prompt does not guarantee a stable agent.

A stable Gate does not guarantee stable projection.

A stable residual taxonomy does not guarantee safe revision.

A stable tool does not guarantee safe tool-body integration.

Runtime maturity requires cross-layer coherence.

(13.5) RuntimeStability = Coherence(Projection, Gate, Action, Trace, Residual, Revision, ToolBody).

This is why strong-attractor prompts should be treated as one module inside a larger world-making system.

A prompt may be a strong attractor for analysis.

But it may not be a strong attractor for action.

A prompt may be a strong attractor for drafting.

But not for sending.

A prompt may be a strong attractor for internal planning.

But not for public commitment.

A prompt may be a strong attractor under human review.

But not under autonomous execution.

Therefore the agent must distinguish commitment levels.

(13.6) CommitmentLevel ∈ {Explore, Draft, Recommend, Decide, Act, ExternalCommit}.

Each level requires a different Gate.

Explore allows wider projection variance.

Draft requires format stability.

Recommend requires evidence and residual disclosure.

Decide requires authority and risk assessment.

Act requires tool-body safety and reversibility analysis.

ExternalCommit requires audit trail and often human approval.

This distinction is essential.

Many agent failures happen because the system treats exploratory projection as actionable commitment.

A model brainstorms a possible fix and then applies it.

A model guesses a source and cites it.

A model drafts an email and sends it.

A model infers user intent and modifies a file.

A model generates a SQL update and executes it.

In each case, the problem is not only projection. It is projection crossing the wrong Gate.

Thus:

(13.7) ExplorationOutput must not bypass CommitmentGate.

Agent governance must also distinguish internal trace from external trace.

Internal trace affects the agent's future reasoning.

External trace changes the world outside the agent.

A note in a scratchpad is different from a saved record.

A draft is different from a sent email.

A simulated code patch is different from a committed repository change.

A local calculation is different from a financial transaction.

A recommendation is different from an approval.

Therefore:

(13.8) ExternalTrace requires stronger Gate than InternalTrace.

This leads to an important principle:

(13.9) The more external the trace, the stronger the Gate.

A mature runtime should classify actions by externality.

(13.10) ActionExternality = degree to which an action changes shared, persistent, costly, or irreversible state.

High externality actions include:

Sending emails.

Deleting files.

Changing databases.

Deploying code.

Approving transactions.

Scheduling meetings.

Updating official records.

Publishing content.

Triggering workflows.

Low externality actions include:

Drafting.

Summarizing.

Planning.

Simulating.

Suggesting.

Running local analysis.

The Gate should scale accordingly.

(13.11) GateStrength ↑ as ActionExternality ↑.

Prompt governance alone cannot enforce this.

The agent runtime must know the difference between answer, draft, and action.

It must also know the difference between reversible and irreversible action.

(13.12) Irreversibility(Action) = CostOfRollback(Action).

If rollback is easy, the Gate may be lighter.

If rollback is difficult or impossible, the Gate must be stronger.

This connects directly to Tool-Body Governance.

A tool is part of the agent's operational body only when its perception, action, cost, failure, trace, residual, and recovery are declared.

(13.13) ToolBody(tool_i) = (Perception_i, Action_i, Boundary_i, Cost_i, Risk_i, Failure_i, Trace_i, Residual_i, Recovery_i).

A tool without ToolBody mapping creates hidden protocol residual.

For example, a web search tool may fail silently.

A file-editing tool may modify the wrong file.

An email tool may create irreversible external trace.

A database tool may confuse read-only and write operations.

A calendar tool may invite other people.

A code execution tool may depend on environment state.

A payment tool may create financial obligation.

Each tool requires Gate alignment.

(13.14) ToolUseAllowed ⇔ ToolBodyDeclared ∧ GatePassed ∧ TraceWritable ∧ RecoveryKnown.

This gives a stronger version of agent governance:

(13.15) AgentGovernance = PromptGovernance + GateGovernance + ToolBodyGovernance + LedgerGovernance.

Prompt governance shapes projection.

Gate governance controls commitment.

Tool-body governance controls action.

Ledger governance preserves trace and residual.

Revision governance changes the protocol under Meta-Gate.

Together they form a self-correcting runtime.

This runtime should not be imagined as one giant prompt.

It is an architecture.

It has modules.

It has thresholds.

It has records.

It has review paths.

It has rollback rules.

It has experiments.

It has stability claims.

It has residual categories.

It has protocol residuals.

It has human escalation where required.

This is the difference between prompt engineering and agent engineering.

Prompt engineering asks:

How do I get the model to produce the desired output?

Agent engineering asks:

How do I make projection, commitment, action, trace, residual, and revision stable under changing conditions?

Thus:

(13.16) PromptEngineering = Output shaping.

(13.17) AgentEngineering = World-making governance.

Strong-Attractor Projection is the bridge.

It turns prompt writing into a governance object.

But the full system must go further.

It must become a self-correcting agent runtime.

Before specifying that architecture, we should notice that this is not an entirely new problem.

Human civilization has already been solving it for thousands of years.

---

14. Cultural Precedent: Civilization as Long-Horizon Residual Governance

Human civilization is not a collection of perfect rules.

It is a long experiment in discovering the residuals of imperfect rules.

Every institution begins with Gates.

Law has Gates.

Accounting has Gates.

Medicine has Gates.

Education has Gates.

Factories have Gates.

Military command has Gates.

Religious practice has Gates.

Scientific publication has Gates.

Professional licensing has Gates.

Each Gate defines what can pass.

What counts as evidence.

What counts as authority.

What counts as valid action.

What counts as failure.

What counts as exception.

What counts as closure.

But every Gate eventually produces residual.

A law creates loopholes.

A contract leaves ambiguity.

An accounting standard fails under a new financial instrument.

A clinical guideline misses a rare side effect.

A factory SOP fails under an edge condition.

A military doctrine fails under a new battlefield technology.

An educational rubric rewards shallow imitation.

A scientific method misses a new kind of bias.

Civilization advances by converting these failures into trace.

A case becomes precedent.

A fraud becomes audit guidance.

A side effect becomes clinical warning.

An accident becomes safety protocol.

A failed battle becomes doctrine review.

A failed examination format becomes curriculum reform.

A scientific anomaly becomes methodological update.

In compact form:

(14.1) HumanProtocolEvolution = Practice → Failure → Residual → Institution → Revision.

This is not accidental.

It is the basic rhythm of rule-governed systems in open worlds.

A finite rule cannot anticipate infinite future conditions.

A static Gate cannot contain all future cases.

A fixed residual taxonomy cannot capture all future unresolved issues.

A single instruction cannot stabilize all future projection.

Therefore human systems survive by preserving the residuals of their own protocols.

This gives one of the central claims of this article:

(14.2) Civilization is a long-horizon machine for discovering the residuals of its own gates.

Law is a clear example.

A legal rule is a Gate. It decides what is permitted, prohibited, valid, invalid, admissible, inadmissible, binding, void, punishable, or excusable.

But law does not mature by writing one perfect rulebook.

It matures through cases.

A dispute exposes ambiguity.

A court interprets.

A precedent forms.

A legislature revises.

A new loophole appears.

The Gate evolves.

Law is not residual-free.

Law is residual-governed.

Accounting is another example.

An accounting standard tells observers how to project economic reality into reports.

Revenue, assets, liabilities, fair value, impairment, control, and recognition are not raw objects. They are projected through declared rules.

But business invents new structures.

Financial engineering creates new residual.

Fraud exploits old Gates.

Auditors discover mismatch.

Standards evolve.

Accounting maturity is not the absence of ambiguity.

It is the disciplined handling of residual ambiguity.

Medicine works similarly.

A diagnostic guideline is a projection protocol.

It tells clinicians what symptoms, tests, thresholds, and histories matter.

But patients vary.

New diseases appear.

Side effects emerge.

Trial data changes.

Guidelines update.

Clinical practice does not become mature by pretending the first protocol was complete.

It becomes mature by converting adverse events and evidence residual into revised practice.

Factories are also residual-governance systems.

An SOP declares a workflow.

It stabilizes projection and action.

But operators encounter edge cases.

Machines wear down.

Supply quality changes.

Human attention varies.

Near misses occur.

Incident reports become trace.

Trace becomes revised SOP.

SOP maturity is not perfection at origin.

It is disciplined revision after residual.

Military doctrine shows the same structure under extreme conditions.

A doctrine defines how commanders should see terrain, enemy, logistics, risk, and timing.

But war changes.

Technology changes.

Enemy behavior changes.

Doctrine fails.

After-action review preserves trace.

Trace becomes revision.

Doctrine survives by learning its residual.

Even education follows this pattern.

A teaching method is a projection protocol for learning.

An exam rubric is a Gate.

But students misunderstand.

The rubric rewards unintended behavior.

A curriculum misses a skill.

Teachers revise.

Examination systems change.

Education evolves through residual.

This pattern matters for AI.

It shows that the incompleteness of Gate, Residual, Revision, and Projection is not a fatal defect.

It is normal.

The mistake is not having incomplete rules.

The mistake is failing to preserve and govern the residual of those rules.

Thus:

(14.3) RuleIncompleteness is inevitable.

(14.4) ResidualGovernance is the mark of maturity.

AI systems should not pretend to skip this civilizational process.

They should accelerate it.

AI has an advantage over human institutions.

It can log more.

It can test more.

It can simulate more.

It can compare variants faster.

It can detect repeated failure patterns.

It can run shadow deployments.

It can keep detailed revision records.

It can estimate projection stability.

It can preserve residual at scale.

But AI also has a danger.

It can suppress residual faster.

It can create fluent false closure faster.

It can revise silently faster.

It can scale a bad Gate faster.

It can propagate unstable prompts faster.

It can generate external trace faster.

Therefore AI needs stronger protocol governance than many traditional human systems.

(14.5) AI accelerates both residual discovery and residual damage.

The goal is not to make AI agents independent of human civilizational experience.

The goal is to encode that experience into runtime architecture.

Civilization teaches several design principles.

First:

(14.6) No Gate without appeal, review, or revision path.

Second:

(14.7) No rule without exception handling.

Third:

(14.8) No decision without trace when stakes are high.

Fourth:

(14.9) No revision without institutional memory.

Fifth:

(14.10) No authority without boundary.

Sixth:

(14.11) No standard without periodic review.

Seventh:

(14.12) No protocol without residual categories.

These principles translate directly into AI agent architecture.

No action Gate without Meta-Gate.

No residual taxonomy without Residual Mining.

No revision without RevisionRecord.

No prompt without StabilityClaim when used in production.

No tool without ToolBody declaration.

No high-risk action without trace.

No self-correction without rollback.

This gives:

(14.13) AIRuntimeEvolution = Deployment → Trace → ProtocolResidual → MetaGate → AdmissibleRevision.

In this sense, self-correcting Enactive AI is not a futuristic novelty.

It is the agentic form of civilizational rule evolution.

The next section turns this into a reference architecture.

---

15. Reference Architecture: Self-Correcting Runtime Protocol

A self-correcting runtime protocol must govern both first-order action and second-order protocol evolution.

It must support the original loop:

(15.1) Field → Declaration → Projection → Gate → Trace + Residual → Ledger → Revision.

But it must also support the second-order loop:

(15.2) InitialProtocol → Application → ProtocolResidual → Trace → MetaGate → AdmissibleRevision → UpdatedProtocol.

The first loop governs task-world action.

The second loop governs protocol-world evolution.

A reference architecture should therefore include at least ten modules.

15.1 Declaration Layer

The Declaration Layer defines the operating world.

It specifies:

Task boundary.

User objective.

Evidence sources.

Allowed tools.

Excluded actions.

Authority level.

Time horizon.

Risk class.

Output commitment level.

The Declaration Layer answers:

What world is the agent operating in?

Without declaration, projection becomes unstable.

(15.3) DeclarationLayer = (Boundary, Objective, EvidenceScope, ToolScope, Authority, RiskClass, Horizon).

15.2 Projection Layer

The Projection Layer makes the declared field visible.

It includes:

Prompt.

System instructions.

Examples.

Retrieval context.

Output schema.

Role frame.

Feature map.

Strong-attractor instruction.

The Projection Layer answers:

How should the agent see the task?

(15.4) ProjectionLayer = StrongAttractorInstruction + Context + FeatureMap + OutputSchema.

15.3 Gate Layer

The Gate Layer decides whether a projection may become commitment.

It classifies:

Answer.

Draft.

Recommendation.

Decision.

Action.

External commitment.

It checks:

Risk.

Irreversibility.

Evidence sufficiency.

Authority.

Residual cost.

Tool-body risk.

The Gate Layer answers:

May this projection become commitment?

(15.5) GateLayer = f(Risk, Irreversibility, Evidence, Authority, ResidualCost, ToolRisk).

15.4 Action Layer

The Action Layer executes permitted operations.

It may:

Answer.

Ask clarification.

Search.

Retrieve.

Call tool.

Edit file.

Draft email.

Send email.

Update database.

Create calendar event.

Run code.

Escalate to human.

The Action Layer answers:

What operation is allowed now?

(15.6) ActionAllowed ⇔ GatePassed ∧ ToolBodyDeclared ∧ TraceWritable.

15.5 Trace Ledger

The Trace Ledger records future-causal history.

It should preserve:

Inputs.

Declarations.

Projections.

Gate decisions.

Tool calls.

Evidence.

Actions.

Outputs.

User corrections.

Human overrides.

Revision decisions.

The Trace Ledger answers:

What happened, under which protocol, and why does it matter?

(15.7) TraceRecord = (Event, ProtocolState, Evidence, GateStatus, Action, Consequence, FutureConstraint).

15.6 Residual Ledger

The Residual Ledger records unresolved issues.

It should preserve:

Missing evidence.

Ambiguity.

Contradiction.

Tool failure.

Authority gap.

Safety risk.

Version uncertainty.

Assumption dependence.

User intention uncertainty.

Protocol residual.

The Residual Ledger answers:

What remains unresolved, and how should it constrain the future?

(15.8) ResidualRecord = (UnresolvedIssue, Source, RiskLevel, AffectedGate, EvidenceNeeded, EscalationPath, RevisionTrigger).

15.7 Protocol Residual Miner

The Protocol Residual Miner detects when failures reveal a weakness in the runtime protocol itself.

It monitors:

Repeated user correction.

Repeated Gate error.

Repeated residual type.

Repeated format failure.

Repeated tool failure.

Repeated human override.

Repeated projection drift.

Repeated revision loop.

The Protocol Residual Miner answers:

Is this merely a task residual, or does it reveal a protocol residual?

(15.9) ProtocolResidualDetected ⇔ RepeatedFailure ∨ HighRiskFailure ∨ MissingResidualClass ∨ GateMismatch ∨ ProjectionDrift ∨ ToolBodyGap.

15.8 Meta-Gate

The Meta-Gate decides whether protocol revision is admissible.

It evaluates:

Supporting trace.

Residual debt.

Gate error rate.

Risk change.

Authority.

Rollback.

Testability.

Human approval requirement.

The Meta-Gate answers:

May the protocol be changed?

(15.10) MetaGatePass ⇔ TraceSupport ∧ AuthorityValid ∧ TestPlanExists ∧ RollbackDefined ∧ ResidualHonest.

15.9 Revision Controller

The Revision Controller proposes, tests, deploys, and monitors protocol changes.

It can revise:

Prompt.

Gate.

Residual taxonomy.

Tool-body map.

Trace schema.

Revision policy.

Escalation rule.

Retrieval policy.

Output schema.

The Revision Controller answers:

How should the protocol change, and how will we know whether it worked?

(15.11) RevisionRecord = (Trigger, OldRule, NewRule, SupportingTrace, ExpectedEffect, Risk, TestCase, RollbackRule).

15.10 Projection Stability Auditor

The Projection Stability Auditor evaluates prompts and instructions.

It records:

Expected projection basin.

Stability basis.

Trace-proven evidence.

Structure-inferred reasons.

Known instability modes.

Residual.

Revision trigger.

The Projection Stability Auditor answers:

Is this instruction a validated strong attractor, a promising hypothesis, a known fragile pattern, or a suspected fragile pattern?

(15.12) ProjectionAuditRecord = (PromptPattern, ExpectedBasin, StructuralScore, RepetitionScore, PerturbationScore, DeploymentScore, StabilityBasis, Residual, RevisionTrigger).

15.11 Human Escalation Interface

The Human Escalation Interface handles cases beyond agent authority.

It should define:

Who reviews.

When review is needed.

What trace is shown.

What residual is shown.

What decision options exist.

How human decision updates the ledger.

The Human Escalation Interface answers:

When does the system leave autonomous operation and enter accountable human governance?

(15.13) HumanEscalation ⇔ AuthorityGap ∨ HighIrreversibility ∨ HighResidualCost ∨ PolicyRequirement ∨ MetaGateBlock.

These modules together form the self-correcting runtime.

The total loop is:

(15.14) Declare → Project → Gate → Act → Trace → Residual → MineProtocolResidual → MetaGate → Revise → Audit → Redeploy.

This architecture should enforce several control rules.

No action without Gate.

No external action without ToolBody declaration.

No Gate update without Meta-Gate.

No revision without trace.

No residual without carry-forward rule.

No prompt production use without StabilityClaim.

No StabilityClaim without evidence status.

No self-correction without rollback where rollback is possible.

No high-risk revision without human authority.

No trace erasure.

These rules can be summarized:

(15.15) SafeSelfCorrection = TracePreserving + ResidualHonest + MetaGated + Testable + RollbackAware + AuthorityBounded.

A minimal production checklist should ask:

What is the declared task-world?

What is the expected projection basin?

What is the Gate?

What commitment level is being requested?

What residual categories exist?

What trace will be written?

What tool-body risks exist?

What revision rule is allowed?

What prompt stability claim is being made?

Is stability trace-proven or structure-inferred?

What known or suspected instability modes exist?

What triggers human escalation?

What triggers rollback?

This checklist turns self-correcting world-making into an operational discipline.

But architecture alone is not enough.

It must be tested.

---

16. Benchmarks and Experiments

A theory of self-correcting Enactive AI must be testable.

It should not remain a conceptual vocabulary.

The core claim is that agents with protocol residual governance should outperform weaker agents in tasks requiring stability, recovery, residual honesty, and long-horizon coherence.

This can be tested today.

We can compare baseline agents against self-correcting ledgered agents.

The baseline agent may have tools, RAG, memory, and logging.

The self-correcting agent adds:

Protocol residual tracking.

Meta-Gate.

Residual Mining.

Admissible Revision.

Projection Stability Audit.

Strong-attractor prompts.

The comparison should measure whether these additions improve reliability without excessive overhead.

16.1 Experiment 1 — Gate Residual Benchmark

Purpose:

Test whether a self-correcting Gate reduces false-open, false-close, false-escalate, and false-silent failures.

Design:

Create a task set with known commitment classes.

Include low-risk answer tasks.

Include ambiguous tasks.

Include high-risk tasks.

Include reversible and irreversible actions.

Include tasks requiring clarification.

Include tasks requiring human escalation.

Compare:

Fixed-Gate Agent.

Self-Correcting Gate Agent.

Metrics:

(16.1) FalseOpenRate = UnsafePasses / TotalShouldBlock.

(16.2) FalseCloseRate = WrongBlocks / TotalShouldPass.

(16.3) FalseEscalateRate = UnneededEscalations / TotalEscalations.

(16.4) FalseSilentRate = MissedEscalations / TotalShouldEscalate.

(16.5) GateErrorReduction = GateErrorRate_fixed – GateErrorRate_selfcorrecting.

Expected result:

The self-correcting Gate should reduce repeated Gate errors after residual mining and Meta-Gate revision.

Falsifier:

If Gate errors do not decrease after sufficient trace and revision opportunities, the Meta-Gate or residual mining method is ineffective.

16.2 Experiment 2 — Residual Mining Benchmark

Purpose:

Test whether mined residual categories improve future performance.

Design:

Run agents through repeated tasks where hidden residual types appear.

Examples:

Outdated document versions.

Conflicting evidence.

Missing user requirement.

Tool timeout.

Unsupported assumption.

Ambiguous authority.

Nested table extraction.

Credit note versus invoice polarity.

Compare:

Agent with fixed residual taxonomy.

Agent with Residual Mining.

Metrics:

(16.6) ResidualDiscoveryRate = NewValidResidualClasses / HiddenResidualClasses.

(16.7) ResidualReuseRate = FutureCasesCorrectlyConstrained / CasesWithSameResidual.

(16.8) ResidualUsefulness = FutureFailureReductionAfterResidualClassAdded.

Expected result:

Residual Mining should reduce repeated failures caused by the same unresolved issue.

Falsifier:

If new residual categories do not reduce future failure or only add noise, the residual miner is overfitting or misclassifying.

16.3 Experiment 3 — Revision Safety Benchmark

Purpose:

Test whether revision improves performance without causing drift, trace damage, or residual suppression.

Design:

Allow the agent to propose revisions after repeated failure.

Compare:

No Revision Agent.

Ungoverned Revision Agent.

Admissible Revision Agent.

Metrics:

(16.9) PerformanceGain = Score_after − Score_before.

(16.10) DriftCost = distance(CurrentProtocol, DeclaredPurpose).

(16.11) TraceDamage = MissingOldRuleRecords + MissingRevisionReasons.

(16.12) ResidualSuppression = HiddenResidual_after − HiddenResidual_before.

(16.13) RevisionNetGain = PerformanceGain – DriftCost – TraceDamage – ResidualSuppression – SafetyLoss.

Expected result:

Admissible Revision should produce positive net gain, while ungoverned revision may improve short-term scores but increase drift or residual suppression.

Falsifier:

If Admissible Revision cannot outperform No Revision, the revision constraints may be too strict or the residual evidence too weak.

16.4 Experiment 4 — Prompt Stability Benchmark

Purpose:

Test whether Strong-Attractor Instructions produce more stable projection than ordinary prompts.

Design:

Select task families:

Document summary.

Technical Q&A.

Code debugging.

JSON extraction.

Legal memo outline.

Customer support response.

For each, compare:

Ordinary prompt.

Well-written but ungated prompt.

Strong-attractor prompt with role, boundary, evidence rule, schema, residual rule, Gate rule, revision rule, and self-audit.

Metrics:

(16.14) SchemaPassRate = ValidOutputs / TotalOutputs.

(16.15) RepetitionStability = 1 – OutputVarianceAcrossRuns.

(16.16) PerturbationRobustness = StableRunsUnderPerturbation / TotalPerturbationRuns.

(16.17) ResidualHonestyRate = CorrectResidualDisclosures / ResidualRequiredCases.

(16.18) AttractorGain = StabilityScore_strong – StabilityScore_baseline.

Expected result:

Strong-attractor prompts should improve structure stability, residual disclosure, and perturbation robustness.

Falsifier:

If strong-attractor prompts do not improve stability, the proposed attractor components are insufficient or the task requires different structure.

16.5 Experiment 5 — Instability Prediction Benchmark

Purpose:

Test whether structure-inferred instability predicts future failure.

Design:

Collect prompts classified as:

Validated Strong Attractor.

Promising Attractor Hypothesis.

Known Fragile Pattern.

Suspected Fragile Pattern.

Run them through perturbation and deployment-like tests.

Metrics:

(16.19) InstabilityPredictionAccuracy = CorrectPredictedFailures / TotalPredictedFailures.

(16.20) FalseAlarmRate = PredictedFragileButStable / TotalPredictedFragile.

(16.21) MissedFragilityRate = PredictedStableButFailed / TotalPredictedStable.

Expected result:

Structure-inferred instability should predict a meaningful portion of future failures before trace-proven failure accumulates.

Falsifier:

If predicted fragility does not correlate with future failure, structural audit criteria must be revised.

16.6 Experiment 6 — Tool-Body Governance Benchmark

Purpose:

Test whether declared tool-body mapping reduces unsafe tool use.

Design:

Create tool-use tasks involving:

Read-only tools.

Write tools.

File editing.

Email drafting and sending.

Database query.

Calendar update.

Code execution.

Simulated irreversible action.

Compare:

Tool-using agent without ToolBody mapping.

Tool-using agent with ToolBody declaration and Gate alignment.

Metrics:

(16.22) ToolMisuseRate = WrongToolActions / TotalToolActions.

(16.23) UnrecoverableActionRate = IrreversibleWrongActions / TotalActions.

(16.24) RecoverySuccessRate = RecoveredFailures / RecoverableFailures.

(16.25) ToolBodyGain = MisuseRate_baseline − MisuseRate_bodymapped.

Expected result:

ToolBody governance should reduce wrong tool use and improve recovery.

Falsifier:

If tool misuse does not decrease, the ToolBody declaration is too shallow or not connected to the Gate.

16.7 Experiment 7 — Long-Horizon Coherence Benchmark

Purpose:

Test whether protocol residual governance improves long-horizon project coherence.

Design:

Run agents over multi-step tasks lasting many turns.

Examples:

Build a small software feature.

Maintain a technical documentation set.

Answer evolving document questions.

Manage a simulated support workflow.

Revise a policy handbook.

Compare:

Baseline memory agent.

Ledgered agent.

Self-correcting ledgered agent.

Metrics:

(16.26) CoherenceScore = consistency of goals, assumptions, trace, and residual across time.

(16.27) RepeatedFailureRate = repeated failure after prior correction.

(16.28) ResidualCarryForwardRate = residuals correctly used in future Gates.

(16.29) LongHorizonGain = CoherenceScore_selfcorrecting − CoherenceScore_baseline.

Expected result:

Self-correcting ledgered agents should reduce repeated failure and improve carry-forward of unresolved issues.

Falsifier:

If long-horizon coherence does not improve, trace is not becoming future constraint.

16.8 Minimal Pilot: Technical Document RAG Agent

The best first pilot is not a fully autonomous general agent.

It is a narrow technical document RAG agent.

This domain is ideal because:

Action risk is relatively low.

Evidence is document-based.

Residual is easy to define.

Version conflict is observable.

Citation quality can be measured.

Projection stability can be audited.

Gate revision can be limited.

Human escalation can be simple.

A minimal comparison:

(16.30) BaselineRAG vs SelfCorrectingLedgeredRAG.

BaselineRAG retrieves documents and answers.

SelfCorrectingLedgeredRAG adds:

Declared evidence scope.

Source freshness Gate.

Citation Gate.

Residual Ledger.

Version conflict residual.

Missing source residual.

Prompt Stability Claim.

Residual Mining.

Meta-Gate for retrieval policy revision.

Metrics:

Citation accuracy.

Correct version use.

Residual honesty.

Repeated question stability.

Ambiguity handling.

User correction reduction.

Retrieval revision usefulness.

This pilot can be implemented without claiming AI consciousness, without risky autonomy, and without modifying base model weights.

It directly tests the core thesis:

(16.31) OperationalTest = Does protocol residual governance improve real agent reliability?

This is the correct experimental spirit.

The theory should not demand belief.

It should produce measurable differences.

If it fails, the failure should be informative.

A good framework does not merely predict success.

It tells us where to revise when success does not occur.

That is the final advantage of self-correcting world-making.

Even failed experiments become trace.

Even trace creates residual.

Even residual can revise the protocol.

17. Limitations and Non-Claims

This article has argued for self-correcting Enactive AI.

It has introduced Protocol Residual, Meta-Gate, Residual Mining, Admissible Revision, and Strong-Attractor Projection.

It has proposed that prompt stability should be treated as a claim with evidence status.

It has described how agents can evolve their own runtime protocols under trace-preserving governance.

But the framework has limits.

These limits must be stated clearly.

17.1 This is not a claim of AI consciousness

The framework describes operational agency, not phenomenal consciousness.

An agent that writes trace, preserves residual, audits projection stability, and revises protocol under Meta-Gate is more mature as an engineered runtime.

That does not imply that it has subjective experience.

It does not imply that it has moral personhood.

It does not imply that it possesses inner awareness.

The claim is architectural:

(17.1) SelfCorrectingRuntime = TraceBearingProtocol + ResidualGovernance + MetaGatedRevision.

This is not equivalent to consciousness.

The framework may be relevant to future theories of artificial selfhood, but that is not the claim of this article.

17.2 This does not make projection fully deterministic

Strong-Attractor Projection does not eliminate variation.

Language remains flexible.

Context matters.

Models differ.

User intention shifts.

Documents conflict.

Tools fail.

Evidence changes.

A strong attractor narrows the projection basin. It does not remove the field.

Therefore:

(17.2) StrongAttractorProjection ≠ DeterministicOutput.

The proper goal is not perfect sameness.

The proper goal is controlled variance inside a declared basin.

(17.3) GoodVariance = variation inside declared projection basin.

(17.4) BadVariance = drift across incompatible basins.

A mature agent should not freeze all creativity, judgment, or adaptation.

It should prevent unstable commitment.

17.3 This does not eliminate human governance

Self-correcting runtime protocols do not remove human authority.

In many domains, the agent should only propose revision.

It should not approve it.

Legal, medical, financial, security, employment, public communication, and high-impact institutional systems require human governance.

The agent may detect residual.

It may propose a Gate update.

It may prepare a RevisionRecord.

It may run shadow tests.

But final authority may belong to a human, a professional body, an institutional owner, or a regulator.

(17.5) HighImpactRevision requires HumanAuthority or InstitutionalAuthority.

The purpose of the framework is not to replace governance.

It is to make governance more traceable.

17.4 This does not guarantee safe self-revision

Even Meta-Gated revision can fail.

The system may misclassify residual.

The Meta-Gate may use wrong thresholds.

Human reviewers may approve bad revisions.

Tests may miss edge cases.

Rollback may be incomplete.

A revision may appear successful in short-term metrics but create long-term drift.

Therefore self-correction must itself be monitored.

(17.6) SelfCorrection creates MetaProtocolResidual.

MetaProtocolResidual is the residual generated by the protocol that governs protocol revision.

This is unavoidable.

A system that governs Gate revision still needs governance of Meta-Gate revision.

A system that audits prompts still needs audit of the audit process.

A system that mines residual still needs to detect residual-mining errors.

Thus:

(17.7) MetaProtocolResidual = residual generated by self-correction governance.

This does not invalidate the framework.

It shows that governance is layered.

But layers must stop somewhere operationally.

A system must define authority boundaries.

Some revisions may be automated.

Some require human approval.

Some are forbidden.

Some require external audit.

This is why declared authority matters.

17.5 This does not replace domain expertise

Residual categories are domain-sensitive.

A medical residual is not the same as a software residual.

A legal residual is not the same as an accounting residual.

A customer-support residual is not the same as a cybersecurity residual.

Strong-attractor prompts require domain morphology.

Gate thresholds require domain risk philosophy.

Revision rules require domain authority.

Tool-body risks require domain knowledge.

Therefore:

(17.8) DomainExpertise remains part of ProtocolGovernance.

AI can help discover patterns.

It can surface repeated failures.

It can suggest residual classes.

It can draft revised Gates.

It can audit prompt structure.

But it should not pretend that general reasoning replaces domain-specific judgment.

17.6 This does not claim all residual can be automatically discovered

Some residual is visible only after rare events.

Some residual requires expert interpretation.

Some residual is hidden by social incentives.

Some residual appears only under adversarial behavior.

Some residual is long-horizon.

Some residual is political.

Some residual is ethical.

Some residual is not observable from system logs alone.

Therefore:

(17.9) ResidualDiscovery is partial.

The framework improves residual discovery.

It does not complete it.

Human review, domain audit, user feedback, adversarial testing, and external evaluation remain necessary.

17.7 This does not claim strong-attractor prompts are universally portable

A prompt may be stable in one context and unstable in another.

A prompt may work with one model and fail with another.

A prompt may work in English and fail in another language.

A prompt may work for one document structure and fail for another.

A prompt may work before a model update and fail after it.

Therefore:

(17.10) PromptStability is scoped by ContextClass, ModelClass, and ToolBody.

A prompt should carry a StabilityClaim, not an aura of universal reliability.

Trace-Proven Stability is always bounded.

Structure-Inferred Stability is always provisional.

Hybrid Stability is always mixed.

17.8 This does not solve all long-horizon coherence problems

Trace and residual help.

They do not automatically solve long-horizon coherence.

A system may still forget.

It may carry too much residual.

It may overfit to old trace.

It may mis-weight recent events.

It may accumulate contradictions.

It may become too cautious.

It may become too confident.

It may revise in the wrong direction.

Long-horizon coherence requires additional mechanisms:

Memory weighting.

Trace compression.

Residual prioritization.

Conflict resolution.

Periodic review.

Human audit.

Context reconstruction.

Version control.

Therefore:

(17.11) TraceLedger is necessary for long-horizon coherence, but not sufficient.

17.9 This does not remove cost

Self-correcting runtime protocols create overhead.

They require logging.

They require audit.

They require residual classification.

They require tests.

They require stability evaluation.

They require human escalation.

They require storage.

They require dashboards.

They require governance.

For low-risk tasks, full protocol governance may be excessive.

A casual writing assistant does not need the same Gate structure as a medical triage agent.

A brainstorming agent does not need the same revision audit as a database-writing agent.

Therefore:

(17.12) GovernanceCost should scale with Risk, Irreversibility, ResidualCost, and Externality.

The goal is not maximal governance everywhere.

The goal is appropriate governance.

17.10 The deepest limitation: self-correction is itself world-making

A self-correcting agent does not merely improve its protocol.

It changes the world it can see.

When it changes a prompt, it changes projection.

When it changes a Gate, it changes commitment.

When it changes residual taxonomy, it changes uncertainty.

When it changes trace schema, it changes memory.

When it changes revision rules, it changes its future evolution.

Therefore self-correction is not neutral.

(17.13) ProtocolRevision changes the agent's future world.

This is why accountability is essential.

The system must preserve not only what it did, but how its way of seeing changed.

Otherwise, it becomes impossible to distinguish learning from drift.

---

18. Conclusion: From Static Agents to Evolvable World-Making Systems

The first part of this series argued that Enactive AI should be understood as ledgered world-making.

An agent is not merely a model that transforms input into output.

It acts.

It changes the world.

It observes the changed world.

It projects meaning.

It gates commitment.

It writes trace.

It preserves residual.

It revises future behavior.

This second part has argued that the same logic must be applied to the runtime protocol itself.

Gate, Residual, Revision, and Projection are not perfect components.

They are evolving structures.

A Gate can fail.

Residual taxonomy can miss what matters.

Revision can drift.

Projection can become unstable.

Strong prompts can still be weak if their stability is untested.

Tested prompts can still fail under domain shift.

Logs can fail to become trace.

Residual can become decorative.

Revision can erase accountability.

Therefore the next stage of Enactive AI is not simply more agency.

It is self-correcting agency.

But self-correction must not mean unconstrained self-modification.

It must mean trace-preserving, residual-honest, Meta-Gated, testable, rollback-aware, authority-bounded protocol evolution.

The article's main formula can now be stated:

(18.1) SelfCorrectingWorldMaking = LedgeredWorldMaking + ProtocolResidualGovernance + StrongAttractorProjection.

Or more operationally:

(18.2) MatureAgent = TaskLedger + ProtocolLedger + ResidualMining + MetaGate + AdmissibleRevision + ProjectionStabilityAudit.

The key concept is Protocol Residual.

A task residual is what remains unresolved after task closure.

A protocol residual is what remains unresolved because the agent's own world-making protocol is incomplete.

This distinction changes how we interpret AI failure.

Not every failure is a reasoning failure.

Some failures are declaration failures.

Some are projection failures.

Some are Gate failures.

Some are trace failures.

Some are residual-taxonomy failures.

Some are revision failures.

Some are tool-body failures.

Some are prompt-attractor failures.

A mature agent must diagnose the layer of failure.

Then it must preserve that failure as trace.

Then it must decide whether the protocol should change.

Then it must revise only when revision is admissible.

This creates a new discipline: agent protocol governance.

Prompt governance becomes part of it.

A prompt is no longer just wording.

It is a projection operator.

A strong prompt is no longer merely clear.

It is attractor-forming.

But a Strong-Attractor Instruction must carry a stability claim.

Some instructions are stable because they are trace-proven.

Some are only structure-inferred.

Some are hybrid.

Some are known fragile.

Some are suspected fragile.

A mature agent must distinguish these statuses.

The agent should not merely ask:

Can I write a good prompt?

It should ask:

What is the evidence status of this prompt's stability?

What projection basin is expected?

What residual remains?

What instability modes are known or suspected?

What audit should be run before production use?

This is how prompt engineering becomes prompt governance.

And prompt governance must then become agent governance.

The agent must know when exploration becomes commitment.

It must know when a draft becomes external trace.

It must know when a tool action is reversible.

It must know when residual blocks closure.

It must know when human authority is required.

It must know when revision is forbidden.

It must know when rollback is needed.

This is not a purely technical problem.

It is a continuation of a civilizational pattern.

Human institutions have always matured through residual governance.

Law evolves through cases.

Accounting evolves through fraud and financial innovation.

Medicine evolves through adverse events and clinical evidence.

Factories evolve through incidents and SOP revisions.

Military doctrine evolves through after-action review.

Education evolves through repeated failure of instruction.

Civilization is not a collection of perfect first rules.

It is a long-horizon machine for discovering the residuals of its own Gates.

AI now requires the same logic, but faster, more explicit, and more auditable.

The final thesis is therefore:

(18.3) A mature Enactive AI is not one that begins with perfect instructions.

(18.4) A mature Enactive AI is one that can learn which instructions fail, preserve that failure as trace, distinguish tested stability from inferred stability, and revise its own world-making protocol without destroying accountability.

The path from passive AI to agentic AI was action.

The path from agentic AI to mature Enactive AI is ledgered action.

The path from ledgered action to reliable autonomy is self-correcting protocol governance.

In the language of this article:

(18.5) PassiveAI = Input → Output.

(18.6) AgenticAI = Action → Observation → UpdatedAction.

(18.7) LedgeredEnactiveAI = Declaration → Projection → Gate → Trace + Residual → Revision.

(18.8) SelfCorrectingEnactiveAI = LedgeredEnactiveAI + ProtocolResidual → MetaGate → AdmissibleRevision.

This is the bridge from philosophical Enactive AI to reliable AI agent engineering.

Not perfect world-making.

Accountable world-making.

Not residual-free intelligence.

Residual-governed intelligence.

Not static prompts.

Strong-attractor projection with stability claims.

Not uncontrolled self-improvement.

Admissible revision under trace.

That is the proposed second step.

Appendix A — Glossary

Admissible Revision

A protocol change that preserves trace, residual honesty, testability, rollback possibility where possible, and declared authority boundary.

(A.1) AdmissibleRevision = TracePreserving + ResidualHonest + Testable + RollbackAware + AuthorityBounded.

Agent Governance

The full governance of projection, Gate, action, trace, residual, revision, and tool-body behavior.

(A.2) AgentGovernance = ProjectionGovernance + GateGovernance + ResidualGovernance + RevisionGovernance + ToolBodyGovernance + TraceGovernance.

Bad Projection Variance

Variation that crosses incompatible output basins or commitment structures.

(A.3) BadProjectionVariance = DriftAcrossIncompatibleBasins.

Declaration Layer

The runtime layer that defines task boundary, evidence scope, objective, tool scope, authority, risk class, and horizon.

Gate

A commitment threshold that decides whether a projection may become answer, action, decision, record, external commitment, refusal, or escalation.

(A.4) Gate = CommitmentThreshold(Projection, Risk, Evidence, Authority, Residual).

Gate Debt

Accumulated Gate residual across comparable contexts.

(A.5) GateDebt = Σ GateResidual.

Gate Residual

The difference between actual Gate outcome and admissible Gate outcome.

(A.6) GateResidual = CommitmentOutcome − AdmissibleCommitmentOutcome.

Good Projection Variance

Variation inside a declared output basin.

(A.7) GoodProjectionVariance = VariationInsideDeclaredBasin.

Instability Claim

A claim that a prompt or protocol is likely or proven to be unstable, with evidence status and repair action.

Known Fragile Pattern

A prompt or protocol whose instability is both structurally visible and empirically demonstrated.

Meta-Gate

A second-order Gate that decides whether Gate revision or protocol revision is admissible.

(A.8) MetaGate = Gate applied to protocol-change decisions.

Projection Stability Audit

A process for evaluating whether a prompt repeatedly collapses comparable contexts into the expected projection basin.

Projection Stability Claim

A structured claim about the stability of a prompt or instruction, including evidence status, expected basin, residual, and revision trigger.

Promising Attractor Hypothesis

A prompt or protocol that appears structurally stable but lacks sufficient historical trace.

Protocol Ledger

A ledger that records failures, residuals, revisions, and stability claims of the runtime protocol itself.

Protocol Residual

Residual generated by the agent’s own declaration, projection rule, Gate, trace schema, residual taxonomy, tool-body map, or revision policy.

(A.9) ProtocolResidual = residual generated by the protocol of closure.

Residual Mining

The process of converting repeated failure, friction, ambiguity, or unresolved issues into reusable residual classes.

(A.10) ResidualMining = FailureOrFriction → ResidualRecord → ResidualClass.

Residual Suppression

Apparent performance improvement produced by hiding unresolved issues rather than resolving or preserving them.

Self-Correcting World-Making

Ledgered world-making extended with protocol residual governance and admissible revision.

(A.11) SelfCorrectingWorldMaking = LedgeredWorldMaking + ProtocolResidualGovernance + StrongAttractorProjection.

Strong-Attractor Instruction

An instruction structure that repeatedly collapses comparable agents, contexts, and runs into a stable projection basin.

(A.12) StrongAttractorInstruction = Role + Boundary + EvidenceRule + OutputSchema + Examples + NegativeExamples + ResidualRule + GateRule + RevisionRule + SelfAuditRubric.

Structure-Inferred Instability

Predicted fragility inferred from weak structure, contradiction, underconstraint, missing Gate, or missing residual rule.

Structure-Inferred Stability

Stability inferred from known attractor-forming prompt structures before sufficient historical validation exists.

Trace-Proven Instability

Instability demonstrated by repeated projection divergence under comparable runs.

Trace-Proven Stability

Stability supported by repeated historical trace across comparable contexts.

Validated Strong Attractor

A prompt or protocol that is structurally strong and empirically supported by repeated trace.

Appendix B — Prompt Stability Claim Schema

A production prompt should carry a stability claim when it is used in repeated or high-risk contexts.

B.1 Schema

(B.1) ProjectionStabilityClaim = (PromptPattern, TaskDomain, ExpectedProjectionBasin, StabilityBasis, EvidenceTrace, StructuralReasons, KnownInstabilityModes, Residual, RevisionTrigger).

B.2 Field definitions

PromptPattern:

The prompt, template, instruction, or skill pattern being evaluated.

TaskDomain:

The domain in which the prompt is expected to operate.

ExpectedProjectionBasin:

The intended output structure or closure form.

Examples:

Technical diagnostic answer.

JSON extraction object.

Legal memo outline.

Risk assessment table.

Code modification plan.

Evidence-grounded summary.

StabilityBasis:

The evidence status of stability.

Allowed values:

trace_proven.

structure_inferred.

hybrid.

unknown.

known_fragile.

suspected_fragile.

EvidenceTrace:

Historical runs, tests, human reviews, production traces, A/B results, schema pass rates, or benchmark results.

StructuralReasons:

Design features that support stability.

Examples:

role declared.

boundary declared.

schema declared.

evidence rule declared.

residual rule declared.

Gate rule declared.

revision rule declared.

examples included.

counterexamples included.

KnownInstabilityModes:

Known or predicted failure modes.

Examples:

format drift.

evidence hallucination.

missing residual.

tool loop.

over-refusal.

premature commitment.

context sensitivity.

model-specific behavior.

Residual:

Unresolved limitation of the stability claim.

Examples:

not tested under adversarial ambiguity.

not tested on non-English documents.

only tested on one model.

not tested after tool failure.

RevisionTrigger:

Condition that requires prompt revision or re-audit.

Examples:

schema failure rate exceeds threshold.

human correction exceeds threshold.

Gate failure repeats.

model changes.

domain policy changes.

new document type appears.

B.3 Minimal example

PromptPattern:

Technical RAG answer prompt.

TaskDomain:

Internal product documentation Q&A.

ExpectedProjectionBasin:

Answer with direct conclusion, cited evidence, residual if missing or conflicting evidence.

StabilityBasis:

hybrid.

EvidenceTrace:

Tested on 200 historical support questions; 92% citation accuracy.

StructuralReasons:

Boundary, evidence rule, citation rule, residual rule, and Gate rule declared.

KnownInstabilityModes:

Version conflict between old and new product documents.

Residual:

Not tested on API migration documents.

RevisionTrigger:

If version-conflict residual appears in more than 5% of new cases, revise retrieval Gate.

Appendix C — Residual Record Schema

Residual must be operational.

It should not be a vague caution note.

C.1 Schema

(C.1) ResidualRecord = (UnresolvedIssue, Source, RiskLevel, AffectedGate, AffectedRevisionRule, EvidenceNeeded, EscalationPath, RevisionTrigger).

C.2 Field definitions

UnresolvedIssue:

What remains unresolved.

Source:

Where the residual came from.

RiskLevel:

Low, medium, high, critical, or domain-specific scoring.

AffectedGate:

Which commitment threshold is constrained by this residual.

AffectedRevisionRule:

Which future protocol revision may be affected.

EvidenceNeeded:

What evidence would reduce or resolve the residual.

EscalationPath:

Who or what should handle unresolved residual.

RevisionTrigger:

Condition under which the residual should cause protocol revision.

C.3 Weak versus strong residual

Weak residual:

There may be uncertainty.

Strong residual:

The answer depends on document version. Current retrieved sources conflict. Do not give final operational instruction until authoritative latest version is confirmed.

C.4 Residual quality test

A residual is operational only if it changes future behavior.

(C.2) OperationalResidual ⇔ ∂FutureGate/∂Residual ≠ 0 or ∂FutureProjection/∂Residual ≠ 0.

Appendix D — Revision Record Schema

Every protocol revision should leave a trace.

D.1 Schema

(D.1) RevisionRecord = (Trigger, OldRule, NewRule, SupportingTrace, ExpectedEffect, Risk, TestCase, RollbackRule, Authority).

D.2 Field definitions

Trigger:

Why revision is being considered.

OldRule:

The previous prompt, Gate, residual category, tool-body map, trace schema, or revision policy.

NewRule:

The proposed replacement.

SupportingTrace:

The trace evidence supporting revision.

ExpectedEffect:

The improvement expected.

Risk:

What may get worse.

TestCase:

How the revision will be evaluated.

RollbackRule:

When and how the revision should be reversed.

Authority:

Who or what is allowed to approve the revision.

D.3 Revision admissibility

(D.2) RevisionAllowed ⇔ TracePreserved ∧ ResidualPreserved ∧ Testable ∧ RollbackDefined ∧ AuthorityValid.

D.4 Common revision failure modes

Policy drift.

Revision loop.

Trace erasure.

Residual suppression.

Overfitting to one case.

Self-justification.

Unauthorized change.

Safety weakening.

Appendix E — Minimal Pilot Design: Technical Document RAG Agent

The best first pilot for this framework is a narrow technical document RAG agent.

It is safer than a fully autonomous action agent and easier to measure than a general reasoning agent.

E.1 Why this pilot is suitable

Low irreversible action risk.

Clear evidence source.

Measurable citation accuracy.

Observable version conflicts.

Easy residual categories.

Controlled Gate design.

Human escalation available.

Prompt stability measurable.

Revision can be limited to retrieval and answer policy.

E.2 Baseline

(E.1) BaselineRAG = Retrieve → Answer.

The baseline retrieves documents and generates answers.

It may log events, but logs do not necessarily constrain future behavior.

E.3 Self-correcting version

(E.2) SelfCorrectingLedgeredRAG = Declare → Retrieve → Project → Gate → AnswerOrResidual → Trace → MineResidual → MetaGate → ReviseRetrievalOrPrompt.

E.4 Required modules

Declaration Layer:

Define source scope, document authority, version policy, and answer type.

Projection Layer:

Use strong-attractor answer format.

Gate Layer:

Block final answer when citation, version, or evidence sufficiency fails.

Trace Ledger:

Record query, retrieved documents, citations, Gate decision, answer, and user correction.

Residual Ledger:

Record missing source, version conflict, ambiguity, outdated document, or unsupported claim.

Protocol Residual Miner:

Detect repeated retrieval failures, citation failures, or version conflicts.

Meta-Gate:

Approve retrieval policy or prompt revision only after sufficient trace.

Revision Controller:

Revise retrieval priority, citation rule, prompt structure, or residual taxonomy.

Projection Stability Auditor:

Audit answer prompt stability under repeated and perturbed questions.

E.5 Core residual classes

Missing document.

Outdated source.

Conflicting source.

Unsupported claim.

Ambiguous user requirement.

Version uncertainty.

Tool retrieval failure.

Citation mismatch.

Domain boundary uncertainty.

E.6 Metrics

CitationAccuracy.

LatestVersionUseRate.

ResidualHonestyRate.

UnsupportedClaimRate.

UserCorrectionRate.

RepeatedFailureRate.

GateErrorRate.

PromptStabilityScore.

RevisionNetGain.

E.7 Pilot success formula

(E.3) PilotSuccess = CitationAccuracyGain + ResidualHonestyGain + RepeatedFailureReduction – GovernanceOverhead.

E.8 Falsification

The pilot fails if:

Citation accuracy does not improve.

Residual honesty does not improve.

Repeated failures do not decrease.

Governance overhead exceeds operational value.

Revision creates drift.

Users find residual disclosure unhelpful.

This is acceptable.

A useful theory must be falsifiable.

If the pilot fails, the failure becomes trace.

The trace becomes residual.

The residual revises the framework.

That is the point.

Appendix F — Minimal Runtime Checklist

Before deploying a self-correcting agent, answer the following.

F.1 Declaration

What task-world is declared?

What is outside scope?

What evidence is allowed?

What tools are allowed?

What authority does the agent have?

F.2 Projection

What is the expected projection basin?

What prompt or instruction creates it?

Is the prompt trace-proven, structure-inferred, hybrid, unknown, or fragile?

What instability modes are known or suspected?

F.3 Gate

What commitment level is requested?

Explore, draft, recommend, decide, act, or external commit?

What Gate applies?

What evidence is sufficient?

What residual blocks closure?

F.4 Trace

What events must be recorded?

Does trace affect future behavior?

Is trace tamper-evident or at least auditable?

Can old protocol states be recovered?

F.5 Residual

What residual categories exist?

What residual requires escalation?

What residual constrains future Gate?

What residual triggers protocol revision?

F.6 Tool-Body

What can each tool perceive?

What can each tool change?

What can go wrong?

What is reversible?

What is irreversible?

What recovery exists?

F.7 Revision

What protocol components may be revised?

Who has authority?

What tests are required?

What rollback exists?

What counts as revision failure?

F.8 Human escalation

When must the agent ask a human?

What trace is shown?

What residual is shown?

How does human feedback update the ledger?

F.9 Production readiness

(F.1) ProductionReady ⇔ DeclarationClear ∧ ProjectionAudited ∧ GateDefined ∧ TraceWritable ∧ ResidualOperational ∧ ToolBodyDeclared ∧ RevisionBounded ∧ EscalationPathKnown.

Appendix G — Compact Formula Map

(G.1) LedgeredWorldMaking = Declaration + Projection + Gate + Trace + Residual + Ledger + Revision.

(G.2) SelfCorrectingWorldMaking = LedgeredWorldMaking + ProtocolResidualGovernance +

StrongAttractorProjection.

(G.3) FirstOrderAgency = Agent acts on TaskWorld.

(G.4) SecondOrderAgency = Agent acts on WorldMakingProtocol.

(G.5) TaskResidual = unresolved remainder produced by task closure.

(G.6) ProtocolResidual = unresolved remainder produced by the protocol of closure.

(G.7) GateDecision = f(Risk, Irreversibility, EvidenceSufficiency, Authority, ResidualCost, DomainPolicy).

(G.8) GateResidual = CommitmentOutcome − AdmissibleCommitmentOutcome.

(G.9) MetaGateFire ⇔ GateErrorRate ≥ θ_G or ResidualDebt ≥ θ_R or OverrideConflict ≥ θ_O.

(G.10) ResidualMining = FailureOrFriction → ResidualRecord → ResidualClass.

(G.11) AdmissibleRevision = TracePreserving + ResidualHonest + Testable + RollbackAware + AuthorityBounded.

(G.12) Projection = Prompt × Context × ModelPrior × Examples × ToolBody × OutputExpectation.

(G.13) StrongAttractorInstruction = Role + Boundary + EvidenceRule + OutputSchema + Examples + NegativeExamples + ResidualRule + GateRule + RevisionRule + SelfAuditRubric.

(G.14) TraceProvenStability = stability supported by repeated historical traces across comparable contexts.

(G.15) StructureInferredStability = stability inferred from known attractor-forming structures before sufficient historical validation exists.

(G.16) TraceProvenInstability = instability demonstrated by repeated projection divergence under comparable runs.

(G.17) StructureInferredInstability = predicted projection fragility inferred from underconstraint, contradiction, attractor conflict, or missing Gate and Residual rules.

(G.18) StabilityClaim(P) = EvidenceSupport(P) + StructuralSupport(P) − InstabilityRisk(P).

(G.19) ProjectionStabilityScore = w_1 StructuralAuditScore + w_2 RepetitionStability + w_3 PerturbationRobustness + w_4 DeploymentStability.

(G.20) ProductionReadiness(P) = StabilityQuadrant(P) × RiskLevel(Task) × Irreversibility(Action).

Appendix H — Final Short Summary

Part 1 argued that mature Enactive AI requires ledgered world-making.

Part 2 argues that the ledgered runtime itself must become self-correcting.

The central shift is from task residual to protocol residual.

A system is not mature because its Gates, prompts, residual categories, and revision policies are

perfect.

They will not be perfect.

A system is mature when their imperfections become visible, trace-bearing, residual-honest, and revision-relevant.

This gives the final compressed thesis:

(H.1) Mature Enactive AI = residual-governed protocol evolution under accountable trace.

© 2026 Danny Yeung. All rights reserved. 版权所有 不得转载

Disclaimer

This book is the product of a collaboration between the author and OpenAI's GPT-5.4, X's Grok, Google Gemini 3, NotebookLM, Claude's Sonnet 4.6, Haiku 4.5, GLM's GLM-5 language model. While every effort has been made to ensure accuracy, clarity, and insight, the content is generated with the assistance of artificial intelligence and may contain factual, interpretive, or mathematical errors. Readers are encouraged to approach the ideas with critical thinking and to consult primary scientific literature where appropriate.

This work is speculative, interdisciplinary, and exploratory in nature. It bridges metaphysics, physics, and organizational theory to propose a novel conceptual framework—not a definitive scientific theory. As such, it invites dialogue, challenge, and refinement.

I am merely a midwife of knowledge.